Majestic Motors Gallery

Session 2023 - 2027

Submitted by:

Ahmad Bin Rashid 2023-CS-53

Supervised by:

Sir Awais Hassan

Ma’am Maida

Course:

CSC-102 Programming Fundamentals

Department of Computer Science

University of Engineering and Technology

Lahore Pakistan

Table of Contents

1. About Majestic Motors Gallery 4
- Contribution: 4
- Key Features: 4

2. Users of Application 4
- Owner: 4
- Manager: 4
- Employee: 5
- Customer: 5

3. Functional Requirements 5
4. Wireframes 7
- Startup Interface: 7
- Owner Menu: 8
- Manager Menu: 8
- Employee Menu: 9

..... 9
- Customer Menu: 9

..... 9
- View Managers Menu: 10

..... 10
- View Vehicles Menu: 10

5. Data Structures (Parallel Arrays) 11
6. Function Prototypes 13
7. Functions Working Flow 17
8. Complete Code 18
9. Weakness in the Application 210
10. Future Directions 210

Table of Figures

Figure 1-Startup Menu 7
Figure 2-Owner Menu 8
Figure 3-Manager Menu 8
Figure 4-Employee Menu 9
Figure 5-Customer Menu 9
Figure 6-View Managers Menu 10
Figure 7-View Vehicles Menu 10

1. About Majestic Motors Gallery

Majestic Motors Gallery, a comprehensive management application designed to streamline processes, enhance customer experiences and provide a secure and efficient operational environment.

- **Contribution:**

Majestic Motors Gallery empowers dealerships to operate strategically, optimizes sales processes and delivers profitability in the competitive automotive industry. Comprehensive reporting and analytics tools provide dealerships with actionable insights into their performance, allowing them to make informed decisions based on strategic planning, targeted marketing and continuous improvement initiatives.

- **Key Features:**

This is very simplified application with detailed specifications according to showroom functionalities and services. Easy-to-use interface designed for seamless navigation and effortless management, provides efficient environment for all types of users.

2. Users of Application

Majestic Motors Gallery provides different access for different type of users. The users are divided into four categories, Owner, Manager, Employee, and Customer. Each user type have access to different types of command related to their need and requirements. User can login and verify their type by inputting the issued username and password for authentication. Furthermore, each client type user has different database connected to it. So that data is stored and kept for specific person.

- **Owner:**

The Owner of the Showroom, which manages the Company, adds Managers and has the access to modify their details. He can view the details of all Managers and Customers working in the Showroom and are registered on the application. He can view the details of all vehicles and products available in the showroom. He can communicate the managers using this application by sending his messages and receiving the messages of Managers. He can also view the total expenditures of the Showroom.

- **Manager:**

The main user who manages the whole showroom and its application, Manager, has access to add and remove the employees and manage their details. He manages the vehicles and associated products. He can see the announcements given by the owner and can send his send message to him. He can communicate with the employees by announcing his orders and by receiving their messages. He can see the registrations done by customers for the vehicle service

and for test drives. He can see the reviews given by customers and the sold item details. He has an access to modify his personal details registered on the application.

- **Employee:**

Employees, the working unit of company, can view details of the vehicles, associated products, and registrations for vehicle service and test drives. He can view the announcements of the manager and can send his message to the managers. He can modify his personal details added in the application.

- **Customer:**

Customers have an easy access to view the vehicles and associated products and buy them following the easy procedure. He can register his vehicle for service. He can register for testing the drives of the most famous and luxurious vehicles available in the showroom. He can modify his personal details. He can also give his reviews or opinions for the owner and the managers.

3. Functional Requirements

Expected functionalities according to types of users:

User Type	***Functionalities***	***Results***
Owner	View Manager Details	View the details of the Managers in tabular form
	Communicate Managers	Send important Announcements and receive the messages by the managers
	Manage Managers	Add and remove the managers and modify their details.
	View Employee Details	View the details of the Employees registered in the application in tabular form
	View Vehicle Details	View the details of the available vehicles in the tabular form
	View Products details	View the details of the associated products in the tabular form
	View Expenses	View the total sum of all the expenditures including salaries of the staff
	View Feedbacks and Opinions	View comments or opinions of the customers

Manager	Important Announcements	View announcements sent by Owner
	Contact to Owner	Send messages to the Owner
	View Vehicle Details	View the details of the available vehicles in the tabular form
	Modify Vehicles	Add, Remove or Update the vehicles
	View Product Details	View the details of the associated products in the tabular form
	Modify Products	Add, Remove or Update the Products
	Sold Vehicles and Products	View tabular details of the Sold Vehicles and Products
	View Employee Details	View the details of the Employees in tabular form
	Manage Employees	Add, remove or update the details of the Employees
	Communicate Employees	Send Announcements and receive messages by Employees
	View Registrations for Vehicle Service	View tabular details of the registrations
	View Registrations for Test Drives	View tabular details of the registrations
	View Comments and Opinions	View comments or opinions of the customers
	Account Settings	Change personal details
Employee	Important Announcements	View announcements sent by Managers
	Contact to Manager	Send messages to the Managers
	View Vehicle Details	View the details of the available vehicles in the tabular form
	View Product Details	View the tabular details of the associated products
	Sold Vehicles and Products	View tabular details of the Sold Vehicles and Products
	View Registrations for Vehicle Service	View tabular details of the registrations
	View Registrations for Test Drives	View tabular details of the registrations
	Account Settings	Change personal details

Customers	View Vehicles	View the details of the available vehicles in the tabular form
	Buy Vehicles	Choose and Buy the Vehicle according to Payment Method
	View Products	View the tabular details of the associated products
	Buy Products	Choose and Buy the Vehicle according to Payment Method
	Register Vehicle for Service	Register your Vehicle for Service
	Register for Test Drive	Register for Testing the Drive of Famous Vehicle
	Add Comments or Opinions	Send Feedbacks or Comments to the Owner and Managers
	Account Settings	Change personal details

4. Wireframes

A few wireframes of this application displayed in Command Line Interface are as follows**:**

- **Startup Interface:**

Through this menu user type will be selected….

Figure 1-Startup Menu

➢ Owner Menu:

Figure 2-Owner Menu

➢ Manager Menu:

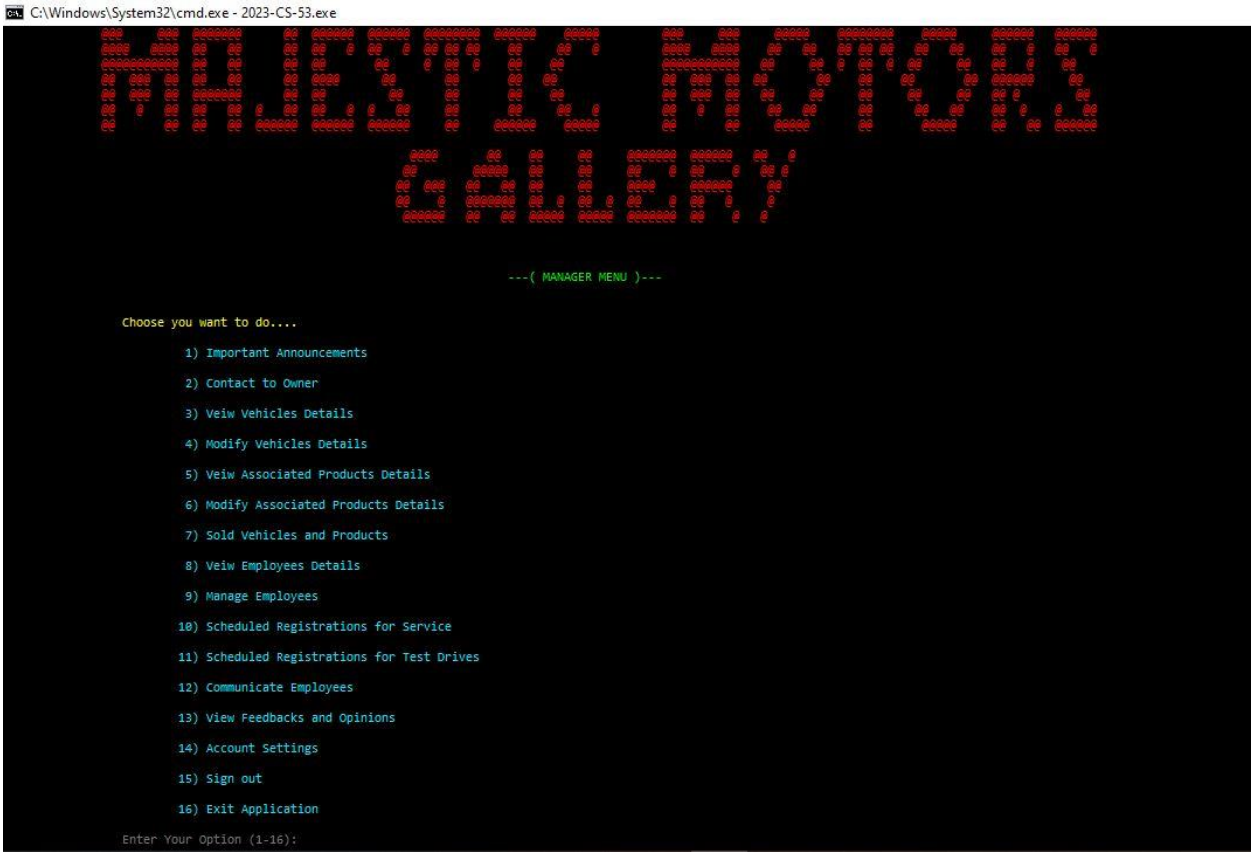


Figure 3-Manager Menu

➢ Employee Menu:

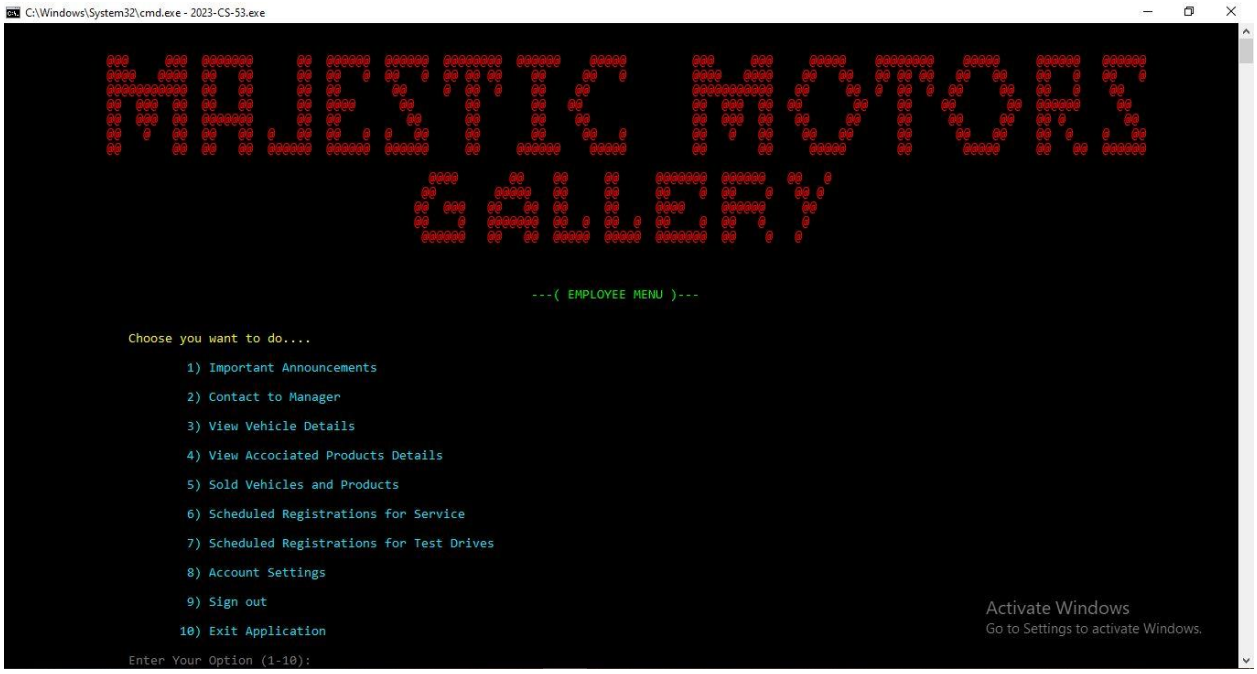


Figure 4-Employee Menu

➢ Customer Menu:

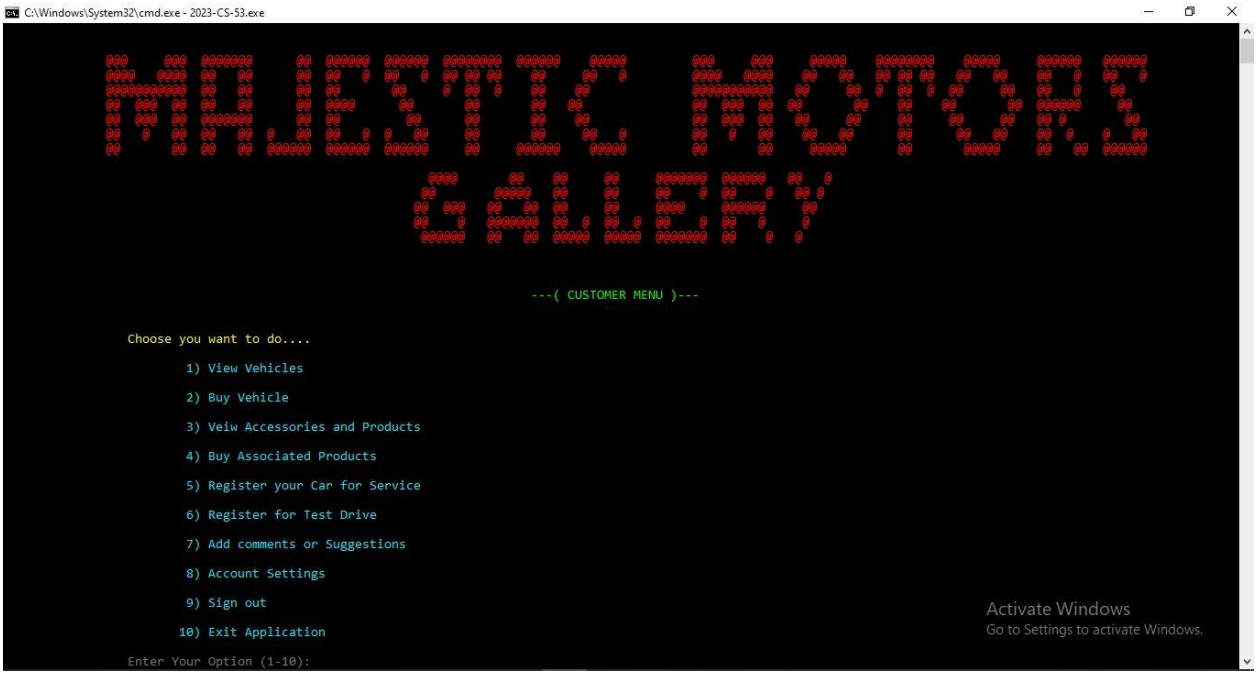


Figure 5-Customer Menu

➢ View Managers Menu:

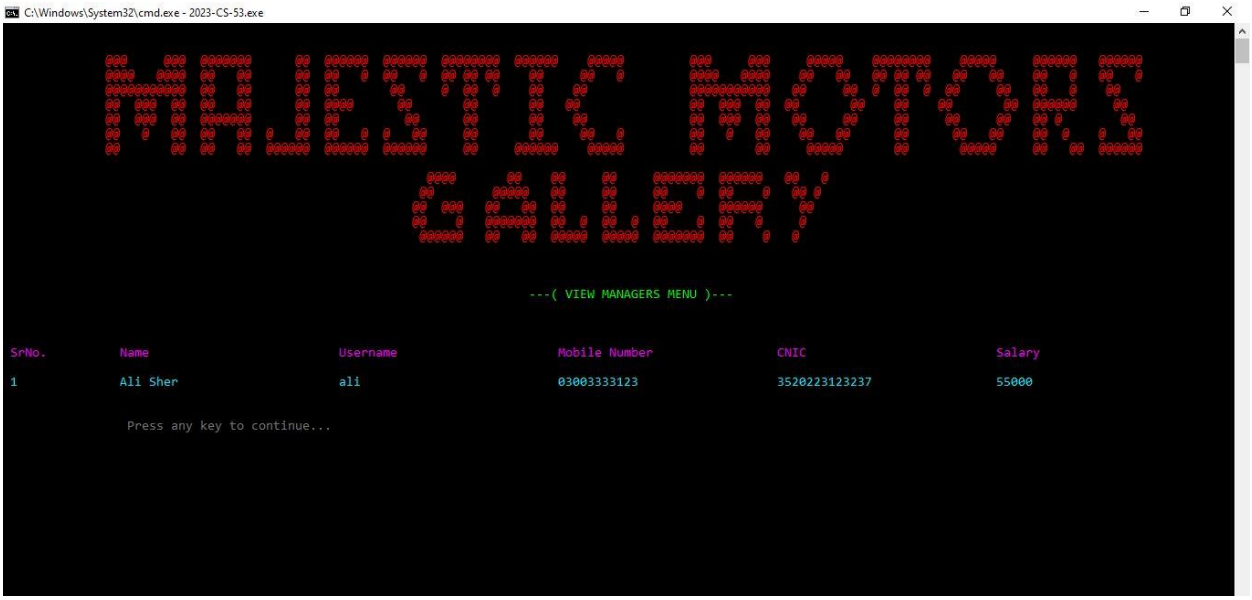


Figure 6-View Managers Menu

➢ View Vehicles Menu:

Figure 7-View Vehicles Menu

5. Data Structures (Parallel Arrays)

```
string managerName[5];
string managerUsername[5];
string managerPassword[5];
string managerPhoneNumber[5];
string managerCNIC[5];
string managerSalary[5];

string employeeName[50];
string employeeUsername[50];
string employeePassword[50];
string employeePhoneNumber[50];
string employeeCNIC[50];
string employeeSalary[50];

string customerName[200];
string customerUsername[200];
string customerPassword[200];
string customerPhoneNumber[200];

string ownerNotices[200];
string managerNotices[200];
string managerName2[200];
string managerUsername2[200];
string managerMessages[200];
string employeeName2[200];
string employeeUsername2[200];
```

```
string employeeMessages[200];

string commentingCustomerName[200];
string commentingCustomerUsername[200];
string customerComments[200];

string customer4ServiceName[200];
string customer4ServiceUsername[200];
string serviceVehicleMake[200];
string serviceVehicleName[200];

string customer4DriveName[200];
string customer4DriveUsername[200];
string customer4DriveVehicle[200];

string vehicleMake[100];
string vehicleName[100];
string vehicleStock[100];
string vehiclePrice[100];

string productName[100];
string productStock[100];
string productPrice[100];

string buyVehicleCustomerName[200];
string buyVehicleCustomerPhoneNumber[200];
string buyVehicleName[200];
string buyVehicleMake[200];
```

```
string buyVehiclePrice[200];
string buyVehiclePayment[200];

string buyProductCustomerName[200];
string buyProductCustomerPhoneNumber[200];
string buyProductName[200];
string buyProductPrice[200];
string buyProductPayment[200];
```

6. Function Prototypes

```
void resizeConsole();
void gotoxy (int, int);
void getUsersData (string, int &, string [], string [], string [], string [], string [], string []);
void getCustomerData (string, int &, string [], string [], string [], string []);
void getVehiclesData (string, int &, string [], string [], string [], string []);
void getProductsData (string, int &, string [], string [], string []);
void getVehicleServiceData (string, int &, string [], string [], string [], string []);
void getTestDerivesData (string, int &, string [], string [], string []);
void getCommentsData (string, int &, string [], string [], string []);
string getField (string, int);
void storeUsersData (string, int, string [], string [], string [], string [], string [], string []);
void storeCustomerData (string, int, string [], string [], string [], string []);
void storeVehiclesData (string, int, string [], string [], string [], string []);
void storeProductsData (string, int, string [], string [], string []);
void storeVehicleServiceData (string, int, string [], string [], string [], string []);
void storeTestDerivesData (string, int, string [], string [], string []);
void storeCommentsData (string, int, string [], string [], string []);
```

```
void printHeader1();
void printHeader2();
void printMessage(string);
void printMenu(string);
void createAccountMessage();
char goBackMessage();
void checkPasscode (string &, string);
void inputCredentials(string &, string &);
bool checkUser (string, string, string [], string [], int);
bool checkAccountEligibility (string [], int);
bool checkEligibility (string [], string, int);
string numberValidation(int, int);
string usernameValidation (int, int);
string passwordValidation (int, int);
string nameValidation (int, int);

string loginMenu1();
string loginMenu2();
string customerLoginMenu();

string ownerMenu();
string _communicateManagersMenu ();
string __ownerAnnouncements (string [], int &);
void __receiveManagerMessagesMenu (string [], string [], string [], int);
void _viewManagersMenu (int, string [], string [], string [], string [], string []);
string _manage_managerMenu();
bool __add_managerMenu(int, string [], string [], string [], string [], string [], string []);
```

```
string __update_managerDetailsMenu(int, string [], string [], string [], string [], string []);
string __remove_managerMenu(int &, string [], string [], string [], string [], string [], string []);
string __update_managerSalaryMenu(int, string [], string [], string [], string []);
void viewExpensesMenu (int, string [], int, string []);
void _viewCommentsMenu (int,  string [], string [], string []);
string managerMenu();
void _showOwnerAnnouncements (string [], int);
string _contactOwnerMenu (int, string [], string [], string, string [], string [], string [], int &);
void _viewVehicle_detailsMenu (int, string [], string [], string [], string []);
string _modifyVehiclesMenu ();
string __addVehicleMenu (int &, string [], string [], string [], string []);
string __updateVehicleMenu (int, string [], string [], string [], string []);
string __removeVehicleMenu (int &, string [], string [], string [], string []);
void _viewProduct_detailsMenu (int, string [], string [], string []);
string _modifyProductMenu ();
string __addProductMenu (int &, string [], string [], string []);
string __updateProductMenu (int, string [], string [], string []);
string __removeProductMenu (int &, string [], string [], string []);
string soldItemsMenu ();
void soldVehiclesMenu (int, string [], string [], string [], string [], string [], string []);
void soldProductsMenu (int, string [], string [], string [], string [], string []);
void _viewEmployeesMenu (int, string [], string [], string [], string [], string []);
string _manage_employeeMenu();
bool __add_employeeMenu(int, string [], string [], string [], string [], string [], string []);
string __update_employeeDetailsMenu(int, string [], string [], string [], string [], string []);
string __remove_employeeMenu(int &, string [], string [], string [], string [], string [], string []);
string __update_employeeSalaryMenu(int, string [], string [], string [], string []);
void _serviceDetailsMenu (int,  string [], string [], string [], string []);
```

void _testDriveDetailsMenu (int, string [], string [], string []);

string _communicateEmployeesMenu ();

string __managerAnnouncements (string [], int &);

void __receiveEmployeeMessagesMenu (string [], string [], string [], int);

string employeeMenu();

void _showManagerAnnouncements (string [], int);

string _contactManagerMenu (int, string [], string [], string, string [], string [], string [], int &);

string accountSettingMenu ();

string changeProfile (int, string [], string, string [], string);

string changeUsername (int, string [], string, string);

string customerMenu();

void _customerViewVehiclesMenu (int, string [], string [], string []);

string buyProductMenu (string);

string chooseVehicleMenu (int, string [], string, string [], string [], int, string [], string [], string [], int, string [], string [], string [], string [], string []);

string paymentMethodMenu1 ();

string paymentMethodMenu2 (int, string [], int, string [], string [], string, string);

string vehicleBillMenu (int, string [], string [], int &, string [], string [], string [], string [], string [], string []);

void _customerViewProductsMenu (int, string [], string []);

string chooseProductMenu (int, string [], string, string [], string [], int, string [], string [], int, string [], string [], string [], string []);

string productBillMenu (int, string [], string [], int &, string [], string [], string [], string [], string []);

string _addCommentsMenu (int, string [], string [], string, int &, string [], string [], string []);

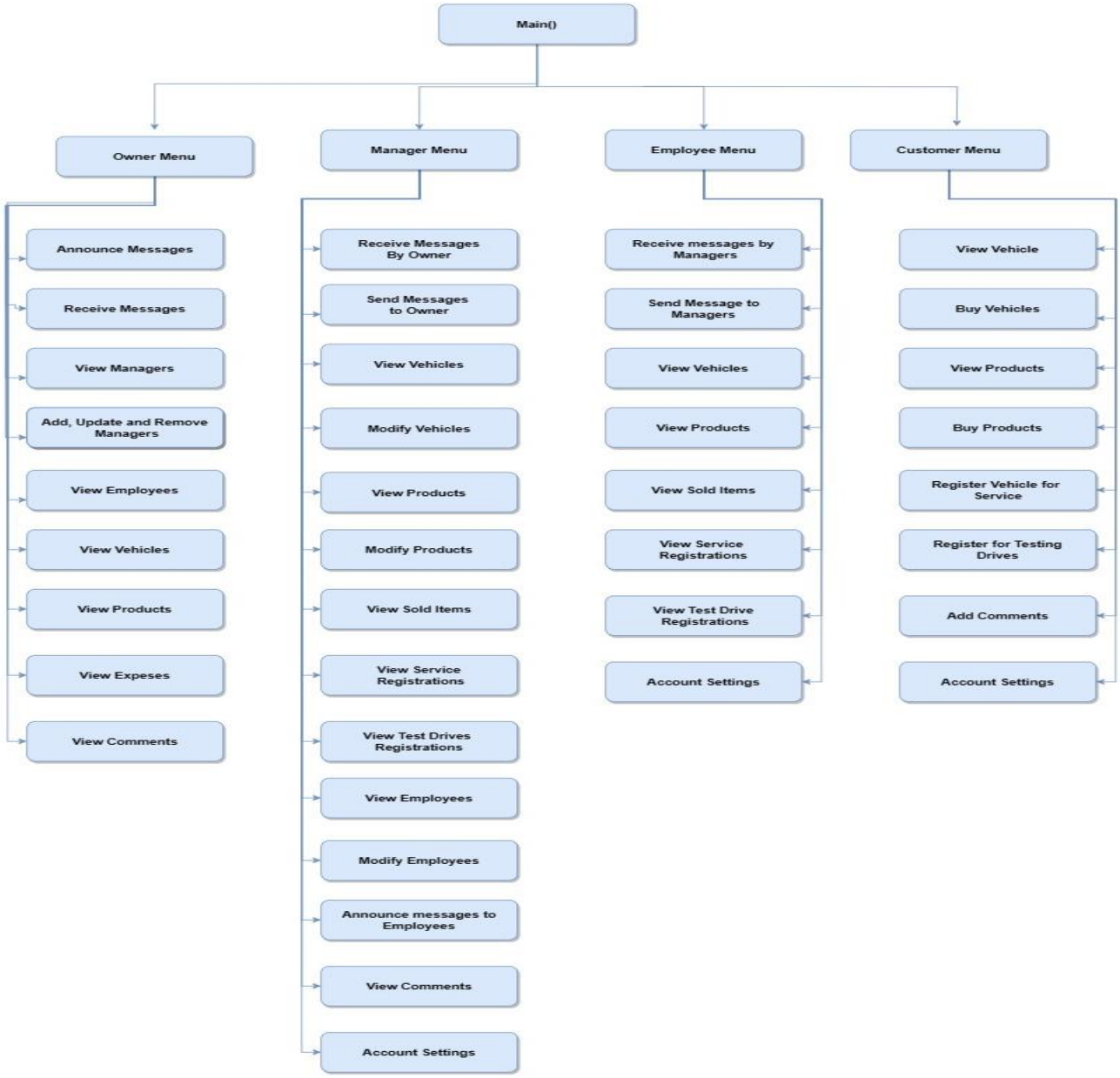
void _serviceRegistrationMenu (int, string [], string [], string, int &, string [], string [], string [], string []);

void _driveRegistrationMenu (int, string [], string [], string, int &, string [], string [], string [], string []);

string customerAccountSettingMenu ();

bool customerSignupMenu(int, string [], string [], string [], string []);

7. Functions Working Flow



8. Complete Code

```
#include <iostream>

#include <fstream>

#include <windows.h>

#include <conio.h>

#include <iomanip>

#include <string>

using namespace std;


void resizeConsole();

void gotoxy (int x, int y);

void getUsersData (string filename, int &userCount, string name[], string username[], string
password[], string phoneNumber[], string CNIC[], string salary[]);

void getCustomerData (string filename, int &userCount, string name[], string username[], string
password[], string phoneNumber[]);

void getVehiclesData (string filename, int &vehicleCount, string make[], string name[], string
stock[], string price[]);

void getProductsData (string filename, int &productCount, string name[], string stock[], string
price[]);

void getVehicleServiceData (string filename, int &customer4ServiceCount, string
customerName[], string customerUsername[], string vehicleMake[], string vehicleName[]);

void getTestDerivesData (string filename, int &customer4TestDriveCount, string
customerName[], string customerUsername[], string vehicleName[]);

void getCommentsData (string filename, int &customerCommentsCount, string
customerName[], string customerUsername[], string comments[]);

string getField (string data, int field);

void storeUsersData (string filename, int userCount, string name[], string username[], string
password[], string phoneNumber[], string CNIC[], string salary[]);

void storeCustomerData (string filename, int userCount, string name[], string username[], string
password[], string phoneNumber[]);
```

```
void storeVehiclesData (string filename, int vehicleCount, string make[], string name[], string stock[], string price[]);

void storeProductsData (string filename, int productCount, string name[], string stock[], string price[]);

void storeVehicleServiceData (string filename, int customer4ServiceCount, string customerName[], string customerUsername[], string vehicleMake[], string vehicleName[]);

void storeTestDerivesData (string filename, int customer4TestDriveCount, string customerName[], string customerUsername[], string vehicleName[]);

void storeCommentsData (string filename, int customerCommentsCount, string customerName[], string customerUsername[], string comments[]);


void printHeader1();

void printHeader2();

void printMessage(string message);

void printMenu(string message);

void createAccountMessage();

char goBackMessage();

void checkPasscode (string &passcode, string message);

void inputCredentials(string &username, string &password);

bool checkUser (string username, string password, string usernameArr[], string passwordArr[], int count);

bool checkAccountEligibility (string usernameArr[], int count);

bool checkEligibility (string array[], string value, int count);

string numberValidation(int maximum, int minimum);

string usernameValidation (int maximum, int minimum);

string passwordValidation (int maximum, int minimum);

string nameValidation (int maximum, int minimum);


string loginMenu1();

string loginMenu2();
```

```
string customerLoginMenu();


string ownerMenu();

string _communicateManagersMenu ();

string __ownerAnnouncements (string ownerNotices[], int &ownerNoticesCount);

void __receiveManagerMessagesMenu (string managerName2[], string managerUsername2[],
string managerMessages[], int managerMessagesCount);

void _viewManagersMenu (int managerCount, string managerName[], string
managerUsername[], string managerPhoneNumber[], string managerCNIC[], string
managerSalary[]);

string _manage_managerMenu();

bool __add_managerMenu(int managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerPhoneNumber[], string
managerCNIC[], string managerSalary[]);

string __update_managerDetailsMenu(int managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerPhoneNumber[], string
managerCNIC[]);

string __remove_managerMenu(int &managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerPhoneNumber[], string
managerCNIC[], string managerSalary[]);

string __update_managerSalaryMenu(int managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerSalary[]);

void viewExpensesMenu (int managerCount, string managerSalary[], int employeeCount, string
employeeSalary[]);

void _viewCommentsMenu (int customerCommentsCount,  string commentingCustomerName[],
string commentingCustomerUsername[], string customerComments[]);


string managerMenu();

void _showOwnerAnnouncements (string ownerNotices[], int ownerNoticesCount);

string _contactOwnerMenu (int managerCount, string managerName[], string
managerUsername[], string temporaryUsername, string managerName2[], string
managerUsername2[], string managerMessages[], int &managerMessagesCount);
```

void _viewVehicle_detailsMenu (int vehicleCount, string vehicleMake[], string vehicleName[], string vehicleStock[], string vehiclePrice[]);

string _modifyVehiclesMenu ();

string __addVehicleMenu (int &vehicleCount, string vehicleMake[], string vehicleName[], string vehicleStock[], string vehiclePrice[]);

string __updateVehicleMenu (int vehicleCount, string vehicleMake[], string vehicleName[], string vehicleStock[], string vehiclePrice[]);

string __removeVehicleMenu (int &vehicleCount, string vehicleMake[], string vehicleName[], string vehicleStock[], string vehiclePrice[]);

void _viewProduct_detailsMenu (int productCount, string productName[], string productStock[], string productPrice[]);

string _modifyProductMenu ();

string __addProductMenu (int &productCount, string productName[], string productStock[], string productPrice[]);

string __updateProductMenu (int productCount, string productName[], string productStock[], string productPrice[]);

string __removeProductMenu (int &productCount, string productName[], string productStock[], string productPrice[]);

string soldItemsMenu ();

void soldVehiclesMenu (int buyVehicleCustomerCount, string buyVehicleCustomerName[], string buyVehicleCustomerPhoneNumber[], string buyVehicleName[], string buyVehicleMake[], string buyVehiclePrice[], string buyVehiclePayment[]);

void soldProductsMenu (int buyProductCustomerCount, string buyProductCustomerName[], string buyProductCustomerPhoneNumber[], string buyProductName[], string buyProductPrice[], string buyProductPayment[]);

void _viewEmployeesMenu (int employeeCount, string employeeName[], string employeeUsername[], string employeePhoneNumber[], string employeeCNIC[], string employeeSalary[]);

string _manage_employeeMenu();

bool __add_employeeMenu(int employeeCount, string employeeUsername[], string employeePassword[], string employeeName[], string employeePhoneNumber[], string employeeCNIC[], string employeeSalary[]);

```
string __update_employeeDetailsMenu(int employeeCount, string employeeUsername[], string
employeePassword[], string employeeName[], string employeePhoneNumber[], string
employeeCNIC[]);

string __remove_employeeMenu(int &employeeCount, string employeeUsername[], string
employeePassword[], string employeeName[], string employeePhoneNumber[], string
employeeCNIC[], string employeeSalary[]);

string __update_employeeSalaryMenu(int employeeCount, string employeeUsername[], string
employeePassword[], string employeeName[], string employeeSalary[]);

void _serviceDetailsMenu (int customer4ServiceCount,  string customer4ServiceName[], string
customer4ServiceUsername[], string serviceVehicleMake[], string serviceVehicleName[]);

void _testDriveDetailsMenu (int customer4DriveCount,  string customer4DriveName[], string
customer4DriveUsername[], string customer4DriveVehicle[]);

string _communicateEmployeesMenu ();

string __managerAnnouncements (string managerNotices[], int &managerNoticesCount);

void __receiveEmployeeMessagesMenu (string employeeName2[], string
employeeUsername2[], string employeeMessages[], int employeeMessagesCount);


string employeeMenu();

void _showManagerAnnouncements (string managerNotices[], int managerNoticesCount);

string _contactManagerMenu (int employeeCount, string employeeName[], string
employeeUsername[], string temporaryUsername, string employeeName2[], string
employeeUsername2[], string employeeMessages[], int &employeeMessagesCount);

string accountSettingMenu ();

string changeProfile (int count, string username[], string temporaryUsername, string data[],
string message);

string changeUsername (int count, string username[], string temporaryUsername, string
message);


string customerMenu();

void _customerViewVehiclesMenu (int vehicleCount, string vehicleMake[], string
vehicleName[], string vehiclePrice[]);

string buyProductMenu (string message);
```

string chooseVehicleMenu (int customerCount, string customerUsername[], string temporaryUsername, string customerName[], string customerPhoneNumber[], int vehicleCount, string vehicleMake[], string vehicleName[], string vehiclePrice[], int buyVehicleCustomerCount, string buyVehicleCustomerName[], string buyVehicleCustomerPhoneNumber[], string buyVehicleName[], string buyVehicleMake[], string buyVehiclePrice[]);

string paymentMethodMenu1 ();

string paymentMethodMenu2 (int count, string productName[], int buyCustomerCount, string buyProductName[], string buyProductPayment[], string message1, string message2);

string vehicleBillMenu (int vehicleCount, string vehicleName[], string vehicleStock[], int &buyVehicleCustomerCount, string buyVehicleCustomerName[], string buyVehicleCustomerPhoneNumber[], string buyVehicleName[], string buyVehicleMake[], string buyVehiclePrice[], string buyVehiclePayment[]);

void _customerViewProductsMenu (int productCount, string productName[], string productPrice[]);

string chooseProductMenu (int customerCount, string customerUsername[], string temporaryUsername, string customerName[], string customerPhoneNumber[], int productCount, string productName[], string productPrice[], int buyProductCustomerCount, string buyProductCustomerName[], string buyProductCustomerPhoneNumber[], string buyProductName[], string buyProductPrice[]);

string productBillMenu (int productCount, string productName[], string productStock[], int &buyProductCustomerCount, string buyProductCustomerName[], string buyProductCustomerPhoneNumber[], string buyProductName[], string buyProductPrice[], string buyProductPayment[]);

string _addCommentsMenu (int customerCount, string customerUsername[], string customerName[], string temporaryUsername, int &customerCommentsCount, string commentingCustomerName[], string commentingCustomerUsername[], string customerComments[]);

void _serviceRegistrationMenu (int customerCount, string customerUsername[], string customerName[], string temporaryUsername, int &customer4ServiceCount, string customer4ServiceName[], string customer4ServiceUsername[], string serviceVehicleMake[], string serviceVehicleName[]);

void _driveRegistrationMenu (int customerCount, string customerUsername[], string customerName[], string temporaryUsername, int &customer4DriveCount, string customer4DriveName[], string customer4DriveUsername[], string customer4DriveVehicle[], string vehicles4Drive[]);

string customerAccountSettingMenu ();

```
bool customerSignupMenu(int customerCount, string customerUsername[], string
customerPassword[], string customerName[], string customerPhoneNumber[]);


HANDLE h = GetStdHandle(STD_OUTPUT_HANDLE);


int main()
{
	const string code ="manager123";
	const string id ="employ123";
	const string ownerCode = "motors123";
	const string ownerUsername = "admin123";
	const string ownerPassword = "123456";

	char backOption;
	string login_Option1 = "";
	string login_Option2 = "";
	string interface1_Option = "";
	string interface2_Option = "";
	string interface3_Option = "";
	string username = "";
	string password = "";
	string passcode = "";

	int managerCount = 0;
	string managerName[5];
	string managerUsername[5];
	string managerPassword[5];
	string managerPhoneNumber[5];
```

```
string managerCNIC[5];
string managerSalary[5];

int employeeCount = 0;
string employeeName[50];
string employeeUsername[50];
string employeePassword[50];
string employeePhoneNumber[50];
string employeeCNIC[50];
string employeeSalary[50];

int customerCount = 0;
string customerName[200];
string customerUsername[200];
string customerPassword[200];
string customerPhoneNumber[200];

bool checkManager;
bool checkEmployee;
bool checkCustomer;
bool eligible;

int ownerNoticesCount = 0;
string ownerNotices[200];

int managerNoticesCount = 0;
string managerNotices[200];
```

```
int managerMessagesCount = 0;
string managerName2[200];
string managerUsername2[200];
string managerMessages[200];

int employeeMessagesCount = 0;
string employeeName2[200];
string employeeUsername2[200];
string employeeMessages[200];

int customerCommentsCount = 0;
string commentingCustomerName[200];
string commentingCustomerUsername[200];
string customerComments[200];

int customer4ServiceCount = 0;
string customer4ServiceName[200];
string customer4ServiceUsername[200];
string serviceVehicleMake[200];
string serviceVehicleName[200];

int customer4DriveCount = 0;
string customer4DriveName[200];
string customer4DriveUsername[200];
string customer4DriveVehicle[200];
string vehicles4Drive[7] = {"Land Rover", "Ford Ranger", "BMW 8 Series Gran Coupe",
"BMW 3 Series Touring", "Mercedes-Benz E-Class Saloon", "Jaguar I-PACE", "Tesla Model
3"};
```

```
int vehicleCount = 0;
string vehicleMake[100];
string vehicleName[100];
string vehicleStock[100];
string vehiclePrice[100];

int productCount = 0;
string productName[100];
string productStock[100];
string productPrice[100];

int buyVehicleCustomerCount = 0;
string buyVehicleCustomerName[200];
string buyVehicleCustomerPhoneNumber[200];
string buyVehicleName[200];
string buyVehicleMake[200];
string buyVehiclePrice[200];
string buyVehiclePayment[200];

int buyProductCustomerCount = 0;
string buyProductCustomerName[200];
string buyProductCustomerPhoneNumber[200];
string buyProductName[200];
string buyProductPrice[200];
string buyProductPayment[200];

string temporaryUsername;
```

```
        getUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername,
managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        getUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

        getCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);

        getVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice);

        getProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);

        getVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount,
customer4ServiceName, customer4ServiceUsername, serviceVehicleMake,
serviceVehicleName);

        getTestDerivesData ("TestDrives_Data.txt", customer4DriveCount,
customer4DriveName, customer4DriveUsername, customer4DriveVehicle);

        getCommentsData ("Comments_Data.txt", customerCommentsCount,
commentingCustomerName, commentingCustomerUsername, customerComments);

        resizeConsole();

        printHeader1();

        while (true)

        {

                printHeader2();

                printMenu("LOGIN");

                login_Option1 = loginMenu1();

                if (login_Option1 == "1" || login_Option1 == "2" || login_Option1 == "3" ||
login_Option1 == "4" || login_Option1 == "5")

                {

                        if (login_Option1 == "1")

                        {

                                while(true)

                                {
```

```
printHeader2();
printMenu("OWNER");
login_Option2 = loginMenu2();

if (login_Option2 == "1" || login_Option2 == "2")
{
    if (login_Option2 == "1")
    {
        printHeader2();
        printMenu("OWNER SIGNIN");

        checkPasscode (passcode, "Enter Owner
Passcode: ");
        if (passcode != ownerCode)
        {
            printMessage("Wrong Passcode.
You are not the Owner!!");
            break;
        }
        inputCredentials(username, password);
        if (username == ownerUsername &&
password == ownerPassword)
        {
            printMessage("\e[1;32mYou
SignedIn Successfully...");

            while (true)
            {
                printHeader2();
```

```
printMenu("OWNER");
interface1_Option =
ownerMenu();

if (interface1_Option == "1"
|| interface1_Option == "2" || interface1_Option == "3" || interface1_Option == "4" ||
interface1_Option == "5" || interface1_Option == "6" || interface1_Option == "7" ||
interface1_Option == "8" || interface1_Option == "9"|| interface1_Option == "10")
{
    if (interface1_Option
== "1")
    {
        while (true)
        {

printHeader2();

printMenu("COMMUNICATE MANAGERS");

interface2_Option =  _communicateManagersMenu();

                                                                    if
(interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3")
                                                                    {

if (interface2_Option == "1")

{

printHeader2();

printMenu("ANNOUNCEMENTS");
```

```
backOption = goBackMessage();
if (backOption == '\b')
{
        continue;
}
printMessage(__ownerAnnouncements (ownerNotices, ownerNoticesCount));
}
else if (interface2_Option == "2")
{
printHeader2();
printMenu("MANAGER MESSAGES");
__receiveManagerMessagesMenu (managerName2, managerUsername2,
managerMessages, managerMessagesCount);
}
else if (interface2_Option == "3")
{
break;
```

```
}
                                                                                    }
                                                                                    else
                                                                                    {
printMessage("Invalid Input. Try again!!");
continue;
                                                                                    }
                                                                                }
                                                                            }
                                                                            else if
(interface1_Option == "2")
                                                                            {
printHeader2();
printMenu("VIEW MANAGERS");
_viewManagersMenu (managerCount, managerName, managerUsername,
managerPhoneNumber, managerCNIC, managerSalary);
                                                                            }
                                                                            else if
(interface1_Option == "3")
                                                                            {
                                                                                while (true)
                                                                                {
printHeader2();
printMenu("MANAGE MANAGER");
```

```
        interface2_Option = _manage_managerMenu();

        if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4" || interface2_Option == "5")
        {
        if (interface2_Option == "1")
        {
        eligible = false;

        printHeader2();
        printMenu("ADD MANAGER");

        backOption = goBackMessage();
        if (backOption == '\b')
        {
                continue;
        }

        eligible = __add_managerMenu(managerCount, managerUsername, managerPassword, managerName, managerPhoneNumber, managerCNIC, managerSalary);
```

```
if (eligible == true)
{
        printMessage("\e[1;32mManager Account Created Successfully...");
        managerCount++;
        storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);
        continue;
}

else if (eligible == false)
{
        printMessage("\e[1;33mUsername already exists. Please try again using other
Credentials...");
        continue;
}
}
else if (interface2_Option == "2")
{
```

```
printHeader2();

printMenu("UPDATE MANAGER");

backOption = goBackMessage();

if (backOption == '\b')

{

        continue;

}

printMessage(__update_managerDetailsMenu(managerCount, managerUsername, managerPassword, managerName, managerPhoneNumber, managerCNIC));

storeUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

}

else if (interface2_Option == "3")

{

printHeader2();

printMenu("REMOVE MANAGER");



backOption = goBackMessage();
```

```
if (backOption == '\b')
{
    continue;
}
printMessage(__remove_managerMenu(managerCount, managerUsername,
managerPassword, managerName, managerPhoneNumber, managerCNIC, managerSalary));
storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);
}
else if (interface2_Option == "4")
{
printHeader2();
printMenu("UPDATE MANAGER");
backOption = goBackMessage();
if (backOption == '\b')
{
    continue;
}
```

```
        printMessage(__update_managerSalaryMenu(managerCount, managerUsername,
managerPassword, managerName, managerSalary));

        storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        }

        else if (interface2_Option == "5")

        {

        break;

        }
                                                                        }
                                                                        else
                                                                        {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                        }
                                                                }
                                                        }
                                                        else if
(interface1_Option == "4")
                                                        {

        printHeader2();

        printMenu("VIEM EMPLOYEES");
```

```
_viewEmployeesMenu (employeeCount, employeeName, employeeUsername, employeePhoneNumber, employeeCNIC, employeeSalary);
}
else if (interface1_Option == "5")
{
printHeader2();
printMenu("VIEW VEHICLES");
_viewVehicle_detailsMenu(vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);
}
else if (interface1_Option == "6")
{
printHeader2();
printMenu("VIEW PRODUCTS");
_viewProduct_detailsMenu (productCount, productName, productStock, productPrice);
}
else if (interface1_Option == "7")
{
printHeader2();
printMenu("VIEW EXPENSES");
```

```
viewExpensesMenu (managerCount, managerSalary, employeeCount, employeeSalary);
}
else if (interface1_Option == "8")
{

printHeader2();

printMenu("VIEW COMMENTS");

_viewCommentsMenu (customerCommentsCount,  commentingCustomerName, commentingCustomerUsername, customerComments);
}
else if (interface1_Option == "9")
{
    break;
}
else if (interface1_Option == "10")
{

storeUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

storeUsersData ("Employees_Data.txt", employeeCount, employeeName, employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC, employeeSalary);

storeCustomerData ("Customers_Data.txt", customerCount, customerName, customerUsername, customerPassword, customerPhoneNumber);
```

```
storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);

storeProductsData ("Products_Data.txt", productCount, productName, productStock, productPrice);

storeVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount, customer4ServiceName, customer4ServiceUsername, serviceVehicleMake, serviceVehicleName);

storeTestDerivesData ("TestDrives_Data.txt", customer4DriveCount, customer4DriveName, customer4DriveUsername, customer4DriveVehicle);

storeCommentsData ("Comments_Data.txt", customerCommentsCount, commentingCustomerName, commentingCustomerUsername, customerComments);
                                        return 0;
                                    }
                                }
                                else
                                {
                                    printMessage("Invalid Input. Try again!!");
                                    continue;
                                }
                            }
                        }
                        else
                        {
                            printMessage("Wrong Credentials!!");
                            continue;
                        }
```

```
                    }
                    else if (login_Option2 == "2")
                    {
                        break;
                    }
                }
                else
                {
                    printMessage("Invalid Input. Try again!!");
                    continue;
                }
            }
        }
        else if (login_Option1 == "2")
        {
            while (true)
            {
                printHeader2();
                printMenu("MANAGER");
                login_Option2 = loginMenu2();

                if (login_Option2 == "1" || login_Option2 == "2")
                {
                    if (login_Option2 == "1")
                    {
                        checkManager = false;

                        printHeader2();
```

```
printMenu("MANAGER SIGNIN");

checkPasscode (passcode, "Enter Company
Passcode: ");
if (passcode != code)
{
        printMessage("Wrong Passcode.
You are not a Manager!!");
        break;
}

inputCredentials(username, password);
checkManager = checkUser (username,
password, managerUsername, managerPassword, managerCount);
if (checkManager == true)
{
        printMessage("\e[1;32mYou
SignedIn Successfully...");
        temporaryUsername = username;
        while (true)
        {
                printHeader2();
                printMenu("MANAGER");
                interface1_Option =
managerMenu();

                if (interface1_Option == "1"
|| interface1_Option == "2" || interface1_Option == "3" || interface1_Option == "4" ||
interface1_Option == "5" || interface1_Option == "6" || interface1_Option == "7" ||
interface1_Option == "8" || interface1_Option == "9"|| interface1_Option == "10" ||
```

```
interface1_Option == "11" || interface1_Option == "12" || interface1_Option == "13" ||
interface1_Option == "14" || interface1_Option == "15" || interface1_Option == "16")
                                                                                {
                                                                                    if (interface1_Option
== "1")
                                                                                    {

        printHeader2();

        printMenu("ANNOUNCEMENTS");

        _showOwnerAnnouncements(ownerNotices, ownerNoticesCount);
                                                                                    }
                                                                                    else if
(interface1_Option == "2")
                                                                                    {

        printHeader2();

        printMenu("COMMUNICATE OWNER");
                                                                                        backOption =
goBackMessage();
                                                                                        if (backOption
== '\b')
                                                                                        {

        continue;
                                                                                        }

        printMessage(_contactOwnerMenu (managerCount, managerName, managerUsername,
temporaryUsername, managerName2, managerUsername2, managerMessages,
managerMessagesCount));
                                                                                    }
```

```
else if (interface1_Option == "3")
{
    printHeader2();
    printMenu("VIEW VEHICLES");
    _viewVehicle_detailsMenu(vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);
}
else if (interface1_Option == "4")
{
    while(true)
    {
    printHeader2();
    printMenu("MODIFY VEHICLES");
    interface2_Option = _modifyVehiclesMenu ();

    if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4")
    {
    if (interface2_Option == "1")
    {
    printHeader2();
```

```
printMenu("ADD VEHICLE");

backOption = goBackMessage();

if (backOption == '\b')

{

        continue;

}

printMessage(__addVehicleMenu (vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice));

storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice);

}

else if (interface2_Option == "2")

{

printHeader2();

printMenu("UPDATE VEHICLE");

backOption = goBackMessage();

if (backOption == '\b')

{
```

```
            continue;

        }

        printMessage(__updateVehicleMenu (vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice));

        storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice);

        }

        else if (interface2_Option == "3")

        {

        printHeader2();

        printMenu("REMOVE VEHICLE");

        backOption = goBackMessage();

        if (backOption == '\b')

        {

            continue;

        }

        printMessage(__removeVehicleMenu (vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice));
```

```
storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);

}

else if (interface2_Option == "4")

{

break;

}
}
else
{

printMessage("Invalid Input. Try again!!");

continue;
}
}
}
else if (interface1_Option == "5")
{

printHeader2();

printMenu("VIEW PRODUCTS");

_viewProduct_detailsMenu (productCount, productName, productStock, productPrice);
}
```

```
else if (interface1_Option == "6")
{
    while(true)
    {
        printHeader2();
        printMenu("MODIFY PRODUCTS");
        interface2_Option = _modifyProductMenu ();

        if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4")
        {
            if (interface2_Option == "1")
            {
                printHeader2();
                printMenu("ADD PRODUCT");
                backOption = goBackMessage();
                if (backOption == '\b')
                {
                    continue;
```

```
        }

        printMessage(__addProductMenu (productCount, productName, productStock,
productPrice));

        storeProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);

        }

        else if (interface2_Option == "2")

        {

        printHeader2();

        printMenu("UPDATE PRODUCT");

        backOption = goBackMessage();

        if (backOption == '\b')

        {

                continue;

        }

        printMessage(__updateProductMenu (productCount, productName, productStock,
productPrice));

        storeProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);
```

```
        }

        else if (interface2_Option == "3")

        {

        printHeader2();

        printMenu("REMOVE PRODUCT");

        backOption = goBackMessage();

        if (backOption == '\b')

        {

                continue;

        }

        printMessage(__removeProductMenu (productCount, productName, productStock,
productPrice));

        storeProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);

        }

        else if (interface2_Option == "4")

        {

        break;
```

```
}
                                                                                    }
                                                                                    else
                                                                                    {
printMessage("Invalid Input. Try again!!");
continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "7")
                                                                    {
                                                                            while (true)
                                                                            {
printHeader2();
printMenu("SOLD ITEMS");
interface2_Option = soldItemsMenu ();

                                                                                    if
(interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3")
                                                                                    {
if (interface2_Option == "1")
{
```

```
printHeader2();

printMenu("SOLD VEHICLES");

soldVehiclesMenu (buyVehicleCustomerCount, buyVehicleCustomerName,
buyVehicleCustomerPhoneNumber, buyVehicleName, buyVehicleMake, buyVehiclePrice,
buyVehiclePayment);

}

else if (interface2_Option == "2")

{

printHeader2();

printMenu("SOLD PRODUCTS");

soldProductsMenu (buyProductCustomerCount, buyProductCustomerName,
buyProductCustomerPhoneNumber, buyProductName, buyProductPrice, buyProductPayment);

}

else if (interface2_Option == "3")

{

break;

}

}

else

{
```

```
printMessage("Invalid Input. Try again!!");

continue;
                                                                                    }
                                                                                }
                                                                            }
                                                                            else if
(interface1_Option == "8")
                                                                            {

printHeader2();

printMenu("VIEW EMPLOYEES");

_viewEmployeesMenu (employeeCount, employeeName, employeeUsername,
employeePhoneNumber, employeeCNIC, employeeSalary);
                                                                            }
                                                                            else if
(interface1_Option == "9")
                                                                            {
                                                                                while (true)
                                                                                {

printHeader2();

printMenu("MANAGE EMPLOYEES");

interface2_Option = _manage_employeeMenu();


                                                                                    if
(interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" ||
interface2_Option == "4" || interface2_Option == "5")
```

```
                                                                                {
        if (interface2_Option == "1")
        {
        eligible = false;

        printHeader2();
        printMenu("ADD EMPLOYEE");

        backOption = goBackMessage();
        if (backOption == '\b')
        {
                continue;
        }

        eligible = __add_employeeMenu(employeeCount, employeeUsername,
employeePassword, employeeName, employeePhoneNumber, employeeCNIC, employeeSalary);
        if (eligible == true)
        {
                printMessage("\e[1;32mEmployee Account Created Successfully...");
```

```
            employeeCount++;

            storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

            continue;

        }


        else if (eligible == false)

        {

            printMessage("\e[1;33mUsername already exists. Please try again using other
Credentials...");

            continue;

        }

        }

        else if (interface2_Option == "2")

        {

        printHeader2();

        printMenu("UPDATE EMPLOYEE");

        backOption = goBackMessage();
```

```
if (backOption == '\b')
{
    continue;
}
printMessage(__update_employeeDetailsMenu(employeeCount, employeeUsername, employeePassword, employeeName, employeePhoneNumber, employeeCNIC));
storeUsersData ("Employees_Data.txt", employeeCount, employeeName, employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC, employeeSalary);
}
else if (interface2_Option == "3")
{
printHeader2();
printMenu("REMOVE EMPLOYEE");


backOption = goBackMessage();
if (backOption == '\b')
{
    continue;
```

```
        }

        printMessage(__remove_employeeMenu(employeeCount, employeeUsername,
employeePassword, employeeName, employeePhoneNumber, employeeCNIC,
employeeSalary));

        storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

        }

        else if (interface2_Option == "4")

        {

        printHeader2();

        printMenu("UPDATE SALARY");

        backOption = goBackMessage();

        if (backOption == '\b')

        {

                continue;

        }

        printMessage(__update_employeeSalaryMenu(employeeCount, employeeUsername,
employeePassword, employeeName, employeeSalary));

        storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
```

```
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

        }

        else if (interface2_Option == "5")

        {

        break;

        }
                                                                                    }
                                                                                    else
                                                                                    {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "10")
                                                                    {

        printHeader2();

        printMenu("SERVICE DETAILS");

        _serviceDetailsMenu (customer4ServiceCount,  customer4ServiceName,
customer4ServiceUsername, serviceVehicleMake, serviceVehicleName);
                                                                    }
```

```
else if (interface1_Option == "11")
{
    printHeader2();
    printMenu("TEST DRIVES DETAILS");
    _testDriveDetailsMenu (customer4DriveCount,  customer4DriveName, customer4DriveUsername, customer4DriveVehicle);
}
else if (interface1_Option == "12")
{
    while (true)
    {
    printHeader2();
    printMenu("COMMUNICATE EMPLOYEES");
    interface2_Option =  _communicateEmployeesMenu();

    if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3")
    {
    if (interface2_Option == "1")
    {
    printHeader2();
```

```
printMenu("ANNOUNCEMENTS");

backOption = goBackMessage();

if (backOption == '\b')

{

        continue;

}

printMessage(__managerAnnouncements (managerNotices, managerNoticesCount));

}

else if (interface2_Option == "2")

{

printHeader2();

printMenu("EMPLOYEE MESSAGES");

__receiveEmployeeMessagesMenu (employeeName2, employeeUsername2,
employeeMessages, employeeMessagesCount);

}

else if (interface2_Option == "3")

{
```

```
        break;

        }
                                                                                    }
                                                                                    else
                                                                                    {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "13")
                                                                    {

        printHeader2();

        printMenu("VIEW COMMENTS");

        _viewCommentsMenu (customerCommentsCount,  commentingCustomerName,
commentingCustomerUsername, customerComments);
                                                                    }
                                                                    else if
(interface1_Option == "14")
                                                                    {
                                                                            while (true)
                                                                            {

        printHeader2();
```

```
printMenu("ACCOUNT SETTING");

interface2_Option = accountSettingMenu();;

if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4" || interface2_Option == "5" || interface2_Option == "6")
{

if (interface2_Option == "1")

{

printHeader2();

printMenu("CHANGE NAME");

printMessage(changeProfile (managerCount, managerUsername, temporaryUsername, managerName, "Name"));

storeUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

}

else if (interface2_Option == "2")

{

printHeader2();

printMenu("CHANGE USERNAME");
```

```
        printMessage(changeUsername (managerCount, managerUsername, temporaryUsername, "Username"));

        storeUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        }

        else if (interface2_Option == "3")

        {

        printHeader2();

        printMenu("CHANGE PASSWORD");

        printMessage(changeProfile (managerCount, managerUsername, temporaryUsername, managerPassword, "Password"));

        storeUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        }

        else if (interface2_Option == "4")

        {

        printHeader2();

        printMenu("CHANGE MOBILE NUMBER");

        printMessage(changeProfile (managerCount, managerUsername, temporaryUsername, managerPhoneNumber, "Mobile Number"));
```

```
storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

}

else if (interface2_Option == "5")

{

printHeader2();

printMenu("CHANGE CNIC");

printMessage(changeProfile (managerCount, managerUsername, temporaryUsername,
managerCNIC, "CNIC"));

storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

}

else if (interface2_Option == "6")

{

break;

}
}
else
{

printMessage("Invalid Input. Try again!!");
```

```
        continue;
                                                                                    }
                                                                                }
                                                                            }
                                                                            else if
(interface1_Option == "15")
                                                                            {
                                                                                break;
                                                                            }
                                                                            else if
(interface1_Option == "16")
                                                                            {

        storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

        storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);

        storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice);

        storeProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);

        storeVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount,
customer4ServiceName, customer4ServiceUsername, serviceVehicleMake,
serviceVehicleName);
```

```
storeTestDerivesData ("TestDrives_Data.txt", customer4DriveCount,
customer4DriveName, customer4DriveUsername, customer4DriveVehicle);

storeCommentsData ("Comments_Data.txt", customerCommentsCount,
commentingCustomerName, commentingCustomerUsername, customerComments);
                                                                    return 0;
                                                                }
                                                            }
                                                            else
                                                            {
                                                                printMessage("Invalid
Input. Try again!!");
                                                                continue;
                                                            }
                                                        }
                                                    }
                                                    else
                                                    {
                                                        printMessage("Wrong Credentials.
Try again!!");
                                                        continue;
                                                    }
                                                }
                                                else if (login_Option2 == "2")
                                                {
                                                    break;
                                                }
                                            }
                                            else
```

```
                    {
                        printMessage("Invalid Input. Try again!!");
                        continue;
                    }
                }
            }
            else if (login_Option1 == "3")
            {
                while (true)
                {
                    printHeader2();
                    printMenu("EMPLOYEE");
                    login_Option2 = loginMenu2();

                    if (login_Option2 == "1" || login_Option2 == "2")
                    {
                        if (login_Option2 == "1")
                        {
                            checkManager = false;

                            printHeader2();
                            printMenu("EMPLOYEE SIGNIN");

                            checkPasscode (passcode, "To confirm that
you are an Employee enter ID provided by Manager: ");
                            if (passcode != id)
                            {
                                printMessage("Wrong ID. You are
not an Employee!!");
```

```
                    continue;
                }

                inputCredentials(username, password);
                checkEmployee = checkUser (username,
password, employeeUsername, employeePassword, employeeCount);

                if (checkEmployee == true)
                {
                    printMessage("\e[1;32mYou
SignedIn Successfully...");
                    temporaryUsername = username;
                    while (true)
                    {
                        printHeader2();
                        printMenu("EMPLOYEE");
                        interface1_Option =
employeeMenu();

                        if (interface1_Option == "1"
|| interface1_Option == "2" || interface1_Option == "3" || interface1_Option == "4" ||
interface1_Option == "5" || interface1_Option == "6" || interface1_Option == "7" ||
interface1_Option == "8" || interface1_Option == "9"|| interface1_Option == "10" ||
interface1_Option == "11" || interface1_Option == "12")
                        {
                            if (interface1_Option
== "1")
                            {

        printHeader2();
```

```
        printMenu("ANNOUNCEMENTS");

        _showManagerAnnouncements (managerNotices, managerNoticesCount);
                                                                        }
                                                                        else if
(interface1_Option == "2")
                                                                        {

        printHeader2();

        printMenu("COMMUNICATE MANAGER");
                                                                            backOption =
goBackMessage();
                                                                            if (backOption
== '\b')
                                                                            {

        continue;
                                                                            }

        printMessage(_contactManagerMenu (employeeCount, employeeName,
employeeUsername, temporaryUsername, employeeName2, employeeUsername2,
employeeMessages, employeeMessagesCount));
                                                                        }
                                                                        else if
(interface1_Option == "3")
                                                                        {

        printHeader2();

        printMenu("VIEW VEHICLES");
```

```
_viewVehicle_detailsMenu(vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);
}
else if (interface1_Option == "4")
{
printHeader2();
printMenu("VIEW PRODUCTS");
_viewProduct_detailsMenu (productCount, productName, productStock, productPrice);
}
else if (interface1_Option == "5")
{
while (true)
{
printHeader2();
printMenu("SOLD ITEMS");
interface2_Option = soldItemsMenu ();
if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3")
{
if (interface2_Option == "1")
{
```

```
printHeader2();

printMenu("SOLD VEHICLES");

soldVehiclesMenu (buyVehicleCustomerCount, buyVehicleCustomerName,
buyVehicleCustomerPhoneNumber, buyVehicleName, buyVehicleMake, buyVehiclePrice,
buyVehiclePayment);

}

else if (interface2_Option == "2")

{

printHeader2();

printMenu("SOLD PRODUCTS");

soldProductsMenu (buyProductCustomerCount, buyProductCustomerName,
buyProductCustomerPhoneNumber, buyProductName, buyProductPrice, buyProductPayment);

}

else if (interface2_Option == "3")

{

break;

}

}

else

{
```

```
printMessage("Invalid Input. Try again!!");

continue;
                                                                                    }
                                                                                }
                                                                            }
                                                                            else if
(interface1_Option == "6")
                                                                            {

printHeader2();

printMenu("SERVICE DETAILS");

_serviceDetailsMenu (customer4ServiceCount,  customer4ServiceName,
customer4ServiceUsername, serviceVehicleMake, serviceVehicleName);
                                                                            }
                                                                            else if
(interface1_Option == "7")
                                                                            {

printHeader2();

printMenu("TEST DRIVES DETAILS");

_testDriveDetailsMenu (customer4DriveCount,  customer4DriveName,
customer4DriveUsername, customer4DriveVehicle);
                                                                            }
                                                                            else if
(interface1_Option == "8")
                                                                            {
                                                                                while (true)
```

```
{
    printHeader2();
    printMenu("ACCOUNT SETTING");
    interface2_Option = accountSettingMenu();;

if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4" || interface2_Option == "5" || interface2_Option == "6")
{
    if (interface2_Option == "1")
    {
    printHeader2();
    printMenu("CHANGE NAME");
    printMessage(changeProfile (employeeCount, employeeUsername, temporaryUsername, employeeName, "Name"));
    storeUsersData ("Employees_Data.txt", employeeCount, employeeName, employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC, employeeSalary);
    }
    else if (interface2_Option == "2")
    {
```

```
printHeader2();

printMenu("CHANGE USERNAME");

printMessage(changeUsername(employeeCount, employeeUsername,
temporaryUsername, "Username"));

storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

}

else if (interface2_Option == "3")

{

printHeader2();

printMenu("CHANGE PASSWORD");

printMessage(changeProfile (employeeCount, employeeUsername, temporaryUsername,
employeePassword, "Password"));

storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

}

else if (interface2_Option == "4")

{
```

```
printHeader2();

printMenu("CHANGE MOBILE NUMBER");

printMessage(changeProfile (employeeCount, employeeUsername, temporaryUsername, employeePhoneNumber, "Mobile Number"));

storeUsersData ("Employees_Data.txt", employeeCount, employeeName, employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC, employeeSalary);

}

else if (interface2_Option == "5")

{

printHeader2();

printMenu("CHANGE CNIC");

printMessage(changeProfile (employeeCount, employeeUsername, temporaryUsername, employeeCNIC, "CNIC"));

storeUsersData ("Employees_Data.txt", employeeCount, employeeName, employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC, employeeSalary);

}

else if (interface2_Option == "6")

{
```

```
        break;

        }
                                                                                    }
                                                                                    else
                                                                                    {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "9")
                                                                    {
                                                                            break;
                                                                    }
                                                                    else if
(interface1_Option == "10")
                                                                    {

        storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

        storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);
```

```
        storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice);

        storeProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);

        storeVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount,
customer4ServiceName, customer4ServiceUsername, serviceVehicleMake,
serviceVehicleName);

        storeTestDerivesData ("TestDrives_Data.txt", customer4DriveCount,
customer4DriveName, customer4DriveUsername, customer4DriveVehicle);

        storeCommentsData ("Comments_Data.txt", customerCommentsCount,
commentingCustomerName, commentingCustomerUsername, customerComments);
                                                                return 0;
                                                        }
                                                }
                                                else
                                                {
                                                        printMessage("Invalid
Input. Try again!!");
                                                        continue;
                                                }
                                        }
                                }
                                else
                                {
                                        printMessage("Wrong Credentials.
Try again!!");
                                        continue;
                                }
```

```
                    }
                    else if (login_Option2 == "2")
                    {
                        break;
                    }
                }
                else
                {
                    printMessage("Invalid Input. Try again!!");
                    continue;
                }
            }
        }
        else if (login_Option1 == "4")
        {
            while (true)
            {
                printHeader2();
                printMenu("CUSTOMER LOGIN");
                login_Option2 = customerLoginMenu();

                if (login_Option2 == "1" || login_Option2 == "2" ||
login_Option2 == "3")
                {
                    if (login_Option2 == "1")
                    {
                        checkCustomer = false;
                        printHeader2();
```

```
printMenu("CUSTOMER SIGNIN");
createAccountMessage();
backOption = goBackMessage();
if (backOption == '\b')
{
    continue;
}

inputCredentials(username, password);
checkCustomer = checkUser (username, password, customerUsername, customerPassword, customerCount);
if (checkCustomer == true)
{
    printMessage("\e[1;32mYou SignedIn Successfully...");
    temporaryUsername = username;
    while (true)
    {
        printHeader2();
        printMenu("CUSTOMER");
        interface1_Option = customerMenu();

        if (interface1_Option == "1" || interface1_Option == "2" || interface1_Option == "3" || interface1_Option == "4" || interface1_Option == "5" || interface1_Option == "6" || interface1_Option == "7" || interface1_Option == "8" || interface1_Option == "9"|| interface1_Option == "10" || interface1_Option == "11" || interface1_Option == "12")
        {
            if (interface1_Option == "1")
```

```
{
printHeader2();
printMenu("VIEW VEHICLES");
_customerViewVehiclesMenu (vehicleCount, vehicleMake, vehicleName, vehiclePrice);
}
else if (interface1_Option == "2")
{
while (true)
{
printHeader2();
printMenu("BUY VEHICLE");
interface2_Option = buyProductMenu ("Vehicle");
if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4")
{
if (interface2_Option == "1")
{
printHeader2();
printMenu("CHOOSE VEHICLE");
printMessage(chooseVehicleMenu (customerCount, customerUsername,
```

```
temporaryUsername, customerName, customerPhoneNumber, vehicleCount, vehicleMake,
vehicleName, vehiclePrice, buyVehicleCustomerCount, buyVehicleCustomerName,
buyVehicleCustomerPhoneNumber, buyVehicleName, buyVehicleMake, buyVehiclePrice));

        }

        else if (interface2_Option == "2")

        {

        while (true)

        {

                printHeader2();

                printMenu("PAYMENT METHOD");

                interface3_Option = paymentMethodMenu1 ();

                if (interface3_Option == "1" || interface3_Option == "2" || interface3_Option ==
"3" || interface3_Option == "4")

                {

                        if (interface3_Option == "1")

                        {

                                printHeader2();

                                printMenu("PAYMENT METHOD");

                                printMessage(paymentMethodMenu2 (vehicleCount,
```

```
vehicleName, buyVehicleCustomerCount, buyVehicleName, buyVehiclePayment, "Immediate
Cash Payment", "Vehicle"));

                    }

                    else if (interface3_Option == "2")

                    {

                        printHeader2();

                        printMenu("PAYMENT METHOD");

                        printMessage(paymentMethodMenu2 (vehicleCount,
vehicleName, buyVehicleCustomerCount, buyVehicleName, buyVehiclePayment, "Payment by
Credit Card", "Vehicle"));

                    }

                    else if (interface3_Option == "3")

                    {

                        printHeader2();

                        printMenu("PAYMENT METHOD");

                        printMessage(paymentMethodMenu2 (vehicleCount,
vehicleName, buyVehicleCustomerCount, buyVehicleName, buyVehiclePayment, "Payment By
Bank", "Vehicle"));

                    }

                    else if (interface3_Option == "4")
```

```
                {
                        break;
                }
        }
        else
        {
                printMessage("Invalid Input. Try again!!");
                continue;
        }
    }
    }
    else if (interface2_Option == "3")
    {
    printHeader2();
    printMenu("PRINT BILL");
    printMessage(vehicleBillMenu (vehicleCount, vehicleName, vehicleStock,
buyVehicleCustomerCount, buyVehicleCustomerName, buyVehicleCustomerPhoneNumber,
buyVehicleName, buyVehicleMake, buyVehiclePrice, buyVehiclePayment));
```

```
        storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName,
vehicleStock, vehiclePrice);

        }

        else if (interface2_Option == "4")

        {

        break;

        }
                                                                                    }
                                                                                    else
                                                                                    {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "3")
                                                                    {

        printHeader2();

        printMenu("VIEW PRODUCTS");

        _customerViewProductsMenu (productCount, productName, productPrice);
                                                                    }
```

```
else if (interface1_Option == "4")
{
    while (true)
    {
        printHeader2();
        printMenu("BUY PRODUCT");
        interface2_Option = buyProductMenu ("Product");
        if (interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" || interface2_Option == "4")
        {
        if (interface2_Option == "1")
        {
        printHeader2();
        printMenu("CHOOSE PRODUCT");
        printMessage(chooseProductMenu (customerCount, customerUsername, temporaryUsername, customerName, customerPhoneNumber, productCount, productName, productPrice, buyProductCustomerCount, buyProductCustomerName, buyProductCustomerPhoneNumber, buyProductName, buyProductPrice));
        }
        else if (interface2_Option == "2")
        {
```

```
while (true)
{
    printHeader2();
    printMenu("PAYMENT METHOD");
    interface3_Option = paymentMethodMenu1 ();
    if (interface3_Option == "1" || interface3_Option == "2" || interface3_Option ==
"3" || interface3_Option == "4")
    {
        if (interface3_Option == "1")
        {
            printHeader2();
            printMenu("PAYMENT METHOD");
            printMessage(paymentMethodMenu2 (productCount,
productName, buyProductCustomerCount, buyProductName, buyProductPayment, "Immediate
Cash Payment", "Product"));
        }
        else if (interface3_Option == "2")
        {
            printHeader2();
```

```
                    printMenu("PAYMENT METHOD");

                    printMessage(paymentMethodMenu2 (productCount,
productName, buyProductCustomerCount, buyProductName, buyProductPayment, "Payment by
Credit Card", "Product"));

            }

            else if (interface3_Option == "3")

            {

                    printHeader2();

                    printMenu("PAYMENT METHOD");

                    printMessage(paymentMethodMenu2 (productCount,
productName, buyProductCustomerCount, buyProductName, buyProductPayment, "Payment By
Bank", "Product"));

            }

            else if (interface3_Option == "4")

            {

                    break;

            }

        }

        else
```

```
            {
                printMessage("Invalid Input. Try again!!");
                continue;
            }
        }
        }
        else if (interface2_Option == "3")
        {
        printHeader2();
        printMenu("PRINT BILL");
        productBillMenu (productCount, productName, productStock,
buyProductCustomerCount, buyProductCustomerName, buyProductCustomerPhoneNumber,
buyProductName, buyProductPrice, buyProductPayment);
        storeProductsData ("Products_Data.txt", productCount, productName, productStock,
productPrice);
        }
        else if (interface2_Option == "4")
        {
        break;
```

```
        }
                                                                                    }
                                                                                    else
                                                                                    {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "5")
                                                                    {

        printHeader2();

        printMenu("SERVICE REGISTRATION");
                                                                            backOption =
goBackMessage();
                                                                            if (backOption
== '\b')
                                                                            {

        continue;
                                                                            }

        _serviceRegistrationMenu (customerCount, customerUsername, customerName,
temporaryUsername, customer4ServiceCount,  customer4ServiceName,
customer4ServiceUsername, serviceVehicleMake, serviceVehicleName);

        storeVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount,
```

```
customer4ServiceName, customer4ServiceUsername, serviceVehicleMake,
serviceVehicleName);
                                                                    }
                                                                    else if
(interface1_Option == "6")
                                                                    {

        printHeader2();

        printMenu("TEST DRIVES REGISTRATION");
                                                                        backOption =
goBackMessage();
                                                                        if (backOption
== '\b')
                                                                        {

        continue;
                                                                        }

        _driveRegistrationMenu (customerCount, customerUsername, customerName,
temporaryUsername, customer4DriveCount, customer4DriveName, customer4DriveUsername,
customer4DriveVehicle, vehicles4Drive);

        storeTestDerivesData ("TestDrives_Data.txt", customer4DriveCount,
customer4DriveName, customer4DriveUsername, customer4DriveVehicle);
                                                                    }
                                                                    else if
(interface1_Option == "7")
                                                                    {

        printHeader2();

        printMenu("ADD COMMENTS");
```

```
                                                                                    backOption =
goBackMessage();
                                                                                    if (backOption
== '\b')
                                                                                    {

        continue;
                                                                                    }

        printMessage(_addCommentsMenu (customerCount, customerUsername,
customerName, temporaryUsername, customerCommentsCount,  commentingCustomerName,
commentingCustomerUsername, customerComments));

        storeCommentsData ("Comments_Data.txt", customerCommentsCount,
commentingCustomerName, commentingCustomerUsername, customerComments);
                                                                        }
                                                                        else if
(interface1_Option == "8")
                                                                        {
                                                                                    while (true)
                                                                                    {

        printHeader2();

        printMenu("ACCOUNT SETTING");

        interface2_Option = customerAccountSettingMenu();;

                                                                                        if
(interface2_Option == "1" || interface2_Option == "2" || interface2_Option == "3" ||
interface2_Option == "4" || interface2_Option == "5")
                                                                                        {
```

```
if (interface2_Option == "1")

{

printHeader2();

printMenu("CHANGE NAME");

printMessage(changeProfile (customerCount, customerUsername, temporaryUsername,
customerName, "Name"));

storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);

}

else if (interface2_Option == "2")

{

printHeader2();

printMenu("CHANGE USERNAME");

printMessage(changeUsername(customerCount, customerUsername,
temporaryUsername, "Username"));

storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);

}

else if (interface2_Option == "3")
```

```
{

printHeader2();

printMenu("CHANGE PASSWORD");

printMessage(changeProfile (customerCount, customerUsername, temporaryUsername,
customerPassword, "Password"));

storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);

}

else if (interface2_Option == "4")

{

printHeader2();

printMenu("CHANGE MOBILE NUMBER");

printMessage(changeProfile (customerCount, customerUsername, temporaryUsername,
customerPhoneNumber, "Mobile Number"));

storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);

}

else if (interface2_Option == "5")

{
```

```
        break;

        }
                                                                                    }
                                                                                    else
                                                                                    {

        printMessage("Invalid Input. Try again!!");

        continue;
                                                                                    }
                                                                            }
                                                                    }
                                                                    else if
(interface1_Option == "9")
                                                                    {
                                                                            break;
                                                                    }
                                                                    else if
(interface1_Option == "10")
                                                                    {

        storeUsersData ("Managers_Data.txt", managerCount, managerName,
managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);

        storeUsersData ("Employees_Data.txt", employeeCount, employeeName,
employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC,
employeeSalary);

        storeCustomerData ("Customers_Data.txt", customerCount, customerName,
customerUsername, customerPassword, customerPhoneNumber);
```

```
storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);

storeProductsData ("Products_Data.txt", productCount, productName, productStock, productPrice);

storeVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount, customer4ServiceName, customer4ServiceUsername, serviceVehicleMake, serviceVehicleName);

storeTestDerivesData ("TestDrives_Data.txt", customer4DriveCount, customer4DriveName, customer4DriveUsername, customer4DriveVehicle);

storeCommentsData ("Comments_Data.txt", customerCommentsCount, commentingCustomerName, commentingCustomerUsername, customerComments);
                                        return 0;
                                    }
                                }
                                else
                                {
                                    printMessage("Invalid Input. Try again!!");
                                    continue;
                                }
                            }
                        }
                        else
                        {
                            printMessage("Wrong Credentials. Try again!!");
                            continue;
                        }
```

```
}
else if (login_Option2 == "2")
{
    eligible = false;
    printHeader2();
    printMenu("CUSTOMER SIGNUP");
    backOption = goBackMessage();
    if (backOption == '\b')
    {
        continue;
    }

    eligible = customerSignupMenu
(customerCount, customerUsername, customerPassword, customerName,
customerPhoneNumber);
    if (eligible == true)
    {
        printMessage("\e[1;32mYou
SignedUp Successfully...");
        customerCount++;
        storeCustomerData
("Customers_Data.txt", customerCount, customerName, customerUsername, customerPassword,
customerPhoneNumber);
        continue;
    }
    else if (eligible == false)
    {
        printMessage("\e[1;33mUsername
already exists. Please try again using other Username...");
        continue;
```

```
                        }
                }
                else if (login_Option2 == "3")
                {
                        break;
                }
        }
        else
        {
                printMessage("Invalid Input. Try again!!");
                continue;
        }
    }
}
else if (login_Option1 == "5")
{
    storeUsersData ("Managers_Data.txt", managerCount, managerName, managerUsername, managerPassword, managerPhoneNumber, managerCNIC, managerSalary);
    storeUsersData ("Employees_Data.txt", employeeCount, employeeName, employeeUsername, employeePassword, employeePhoneNumber, employeeCNIC, employeeSalary);
    storeCustomerData ("Customers_Data.txt", customerCount, customerName, customerUsername, customerPassword, customerPhoneNumber);
    storeVehiclesData ("Vehicles_Data.txt", vehicleCount, vehicleMake, vehicleName, vehicleStock, vehiclePrice);
    storeProductsData ("Products_Data.txt", productCount, productName, productStock, productPrice);
    storeVehicleServiceData ("VehicleService_Data.txt", customer4ServiceCount, customer4ServiceName, customer4ServiceUsername, serviceVehicleMake, serviceVehicleName);
```

```
                                        storeTestDerivesData ("TestDrives_Data.txt",
customer4DriveCount, customer4DriveName, customer4DriveUsername,
customer4DriveVehicle);

                                        storeCommentsData ("Comments_Data.txt",
customerCommentsCount, commentingCustomerName, commentingCustomerUsername,
customerComments);

                                        return 0;

                                }

                        }

                        else

                        {

                                printMessage("Invalid Input. Try again!!");

                                continue;

                        }

                }

}



void printHeader1()

{

                system("cls");

        SetConsoleTextAttribute(h, 12);       // Set text color to red

                cout <<R"(

                                                                (          (

                                                        (       (              )\ )      ))\ )  (

                                                        )\))(   )\     (  (  (()/(` )  /(()/(  )\

                                                        ((_)()((((_)(   )\ )\  /(_))( )(_))(_)|((_)

                                                        (_()((_)\ _ )\ ((_|(_)(_)) (_(_()|_)) )\___

                                                        |  \/  (_)_\(_) | | __/ __||_   _|_ _((/ __|
```

```
| |\/| |/ _ \| || | _|\__ \  | |  | | | (__
|_|  |_/_/ )_\\__/|____|_)_/( |_|(|____| \____|

(      ( /(       ) ( /(  )\ ) )\ )   (       (     )\ ) )\ )     )\ ) ( /(
)\))(  )\())` )  /( )\())(()/((()/(   )\ )    )\   (()/((()/( (  (()/( )\())
((_)()\((_)\  ( )(_)|(_)\  /(_))/(_)) (()/( ((((_)(  /(_))/(_)))\  /(_)|(_)\
(_()((_) ((_)(_(_())  ((_)(_)) (_))    /(_))_)\ _ )\(_)) (_)) ((_)(_))__ ((_)
|  \/  |/ _ \|_   _| / _ \| _ \/ __|  (_)) __(_)_\(_) |  | |  | __| _ \ \ / /
| |\/| | (_) | | |  | (_) |   /\__ \    | (_ |/ _ \ | |__| |__| _||   /\ V /
|_|  |_|\___/  |_|   \___/|_|_\|___/     \___/_/ \_\|_____|_____|___|_|_\ |_|
```

L O A D I N G

)";

```
    cout <<"\t\t\t\t\t\t\t";
    for (int i = 0; i < 25; i++)
    {
        char ch = 219;                // Ascii character to print box
        cout << ch << ch;
        Sleep(60);
    }
```

```
}



void printHeader2()
{
        system("cls");
   SetConsoleTextAttribute(h, 12);      // Set text color to red
        cout << endl << endl;


        cout <<"          @@@    @@@  @@@@@@@@     @@  @@@@@@
@@@@@@  @@@@@@@@@  @@@@@@   @@@@@       @@@    @@@
@@@@@   @@@@@@@@@   @@@@@    @@@@@@  @@@@@@     "<< endl;
        cout <<"          @@@@   @@@@  @@   @@    @@  @@   @  @@   @  @@ @@
@@   @@    @@   @      @@@@   @@@@   @@   @@   @@ @@ @@   @@   @@
@@   @   @@   @     "<< endl;
        cout <<"          @@@@@@@@@@@@  @@   @@    @@  @@     @@    @  @@
@   @@   @@          @@@@@@@@@@@@   @@    @@  @  @@  @  @@    @@
@@   @   @@       "<< endl;
        cout <<"          @@  @@@  @@  @@   @@    @@  @@@@    @@     @@
@@   @@          @@  @@@  @@  @@     @@   @@   @@     @@  @@@@@@
@@       "<< endl;
        cout <<"          @@  @@@  @@  @@@@@@@@    @@  @@      @@    @@
@@   @@          @@  @@@  @@   @@    @@    @@    @@    @@   @@ @     @@
"<< endl;
        cout <<"          @@   @   @@  @@   @@  @   @@  @@   @  @   @@    @@
@@    @@   @      @@   @   @@   @@   @@    @@    @@   @@    @@ @   @   @@
"<< endl;
        cout <<"          @@     @@  @@   @@  @@@@@@  @@@@@@  @@@@@@
@@    @@@@@@   @@@@@      @@     @@    @@@@@     @@     @@@@@
@@   @@  @@@@@@     "<< endl;
        cout << endl;
        cout <<"                                    @@@@     @@   @@    @@
@@@@@@@  @@@@@@   @@   @ " << endl;
```

```
        cout <<"                                        @@      @@@@@@  @@    @@    @@
@  @@    @   @@ @  " << endl;
        cout <<"                                       @@  @@@   @@   @@  @@    @@
@@@@    @@@@@@    @@   " << endl;
        cout <<"                                       @@   @  @@@@@@@@ @@  @  @@  @
@@    @  @@   @    @    " << endl;
        cout <<"                                        @@@@@@@  @@   @@  @@@@@@
@@@@@@  @@@@@@@@  @@    @   @     " << endl;
        cout << endl << endl << endl ;


}



void printMessage(string message)
{
        SetConsoleTextAttribute(h, 9);      // Set text color to blue
        cout << endl;
        cout <<"\t\t\t\t"<< message << endl;
        Sleep(1100);
}



void printMenu(string message)
{
        SetConsoleTextAttribute(h, 10);      // Set text color to Green


        cout <<"\t\t\t\t\t\t\t\t\t---( "<< message <<" MENU )---"<< endl << endl << endl;
}
```

```
void createAccountMessage ()
{
	SetConsoleTextAttribute(h, 15);		// Set text color to white

	cout <<"\t\t If you don't have any account, Please go back and create your account..."<<
endl << endl;
}


char goBackMessage()
{
	char option;
	SetConsoleTextAttribute(h, 6);
	cout <<"\t\t Enter Backspace Key to go back."<< endl;
	cout <<"\t\t Enter any other key to proceed: ";
	option = getch();
	cout << endl << endl;
	return option;
}


void checkPasscode (string &passcode, string message)
{
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t "<< message ;
	getline(cin, passcode);
	cout << endl;
```

```
}


void inputCredentials(string &username, string &password)
{
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter User Name: ";
	username = usernameValidation(10,4);
	cout <<"\t\t\t Enter Password: ";
	password = passwordValidation(10,4);
}


string loginMenu1()
{
	string option;

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t You want to Login as: "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t 1) OWNER"<< endl << endl;
	cout <<"\t\t\t 2) MANAGER"<< endl << endl;
	cout <<"\t\t\t 3) EMPLOYEE"<< endl << endl;
	cout <<"\t\t\t 4) CUSTOMER"<< endl << endl << endl;
	cout <<"\t\t\t 5) Exit Application"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-5): ";
	getline(cin, option);
```

```
        return option;
}


string loginMenu2()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Select one of these: "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) SignIn"<< endl << endl;
        cout <<"\t\t\t 2) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-2): ";
        getline(cin, option);

        return option;
}


string customerLoginMenu()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Select one of these: "<< endl << endl;
```

```
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) SignIn"<< endl << endl;
        cout <<"\t\t\t 2) SignUp"<< endl << endl << endl;
        cout <<"\t\t\t 3) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-3): ";
        getline(cin, option);

        return option;
}



string ownerMenu()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Communicate Managers"<< endl << endl;
        cout <<"\t\t\t 2) Veiw Managers Details"<< endl << endl;
        cout <<"\t\t\t 3) Manage Managers"<< endl << endl;
        cout <<"\t\t\t 4) Veiw Employees Details"<< endl << endl;
        cout <<"\t\t\t 5) Veiw Vehicles Details"<< endl << endl;
        cout <<"\t\t\t 6) Veiw Associated Products Details"<< endl << endl;
        cout <<"\t\t\t 7) View Expenses"<< endl << endl;
        cout <<"\t\t\t 8) View Feedbacks and Opinions"<< endl << endl;
        cout <<"\t\t\t 9) Sign out"<< endl << endl;
```

```
        cout <<"\t\t\t10) Exit Application"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-10): ";
        getline(cin, option);

        return option;
}


string _communicateManagersMenu ()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Announce Important Message"<< endl << endl;
        cout <<"\t\t\t 2) Message received by Managers"<< endl << endl;
        cout <<"\t\t\t 3) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-3): ";
        getline(cin>>ws, option);

        return option;
}


string __ownerAnnouncements (string ownerNotices[], int &ownerNoticesCount)
```

```
{
	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Announce Important Message to Managers: "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Your Announcement: ";
	getline(cin, ownerNotices[ownerNoticesCount]);
	ownerNoticesCount++;

	return "Your Message has been Delivered Successfully...";
}



void __receiveManagerMessagesMenu (string managerName2[], string managerUsername2[],
string managerMessages[], int managerMessagesCount)
{
	if (managerMessagesCount == 0)
	{
		cout << endl << endl;
		SetConsoleTextAttribute(h, 12);
		cout <<"\t\t No messages received by any Manager yet."<< endl;
	}
	else if (managerMessagesCount > 0)
	{
		cout << endl;
		SetConsoleTextAttribute(h, 13);
		cout <<"  "<< setw(7) << left <<"SrNo."<< setw(10) << left <<"Name"<<
setw(15) << left <<"UserName"<< setw(10) << left <<"Message" << endl;
		SetConsoleTextAttribute(h, 11);
		for (int x = 0; x < managerMessagesCount; x++)
```

```
		{
			cout <<"  "<< setw(7) << left << x+1 << setw(10) << left << managerName2[x] << setw(15) << left << managerUsername2[x] << setw(10) << left <<managerMessages[x] << endl;
		}
	}
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to continue...";
	getch();
}


void _viewManagersMenu (int managerCount, string managerName[], string managerUsername[], string managerPhoneNumber[], string managerCNIC[], string managerSalary[])
{
	cout << endl;
	if (managerCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout << endl << endl;
		cout <<"\t\t No Manager Added Yet!!"<< endl;
	}
	else
	{
		SetConsoleTextAttribute(h, 13);
		cout <<" "<< setw(15) << left <<"SrNo."<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Mobile Number"<< setw(30) << left <<"CNIC"<< setw(30) << left <<"Salary"<< endl << endl;
```

```
                SetConsoleTextAttribute(h, 11);
                for (int x = 0; x < managerCount; x++)
                {
                        cout <<" "<< setw(15) << left << x + 1 << setw(30) << left <<
managerName[x] << setw(30) << left << managerUsername[x] << setw(30) << left <<
managerPhoneNumber[x] << setw(30)<< left << managerCNIC[x] << setw(30) << left <<
managerSalary[x] << endl << endl;
                }
        }

        cout << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}



string _manage_managerMenu()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Add an Manager"<< endl << endl;
        cout <<"\t\t\t 2) Update Manager Details"<< endl << endl;
        cout <<"\t\t\t 3) Remove an Manager"<< endl << endl;
        cout <<"\t\t\t 4) Update Manager Salary"<< endl << endl;
        cout <<"\t\t\t 5) Go Back"<< endl << endl;
```

```
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-5): ";
	getline(cin, option);

	return option;
}



bool __add_managerMenu(int managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerPhoneNumber[], string
managerCNIC[], string managerSalary[])
{
	bool eligible;
	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Press Tab Key to Enter..."<< endl << endl;
	SetConsoleTextAttribute(h, 11);

	cout <<"\t\t\t Enter Manager Full Name: ";
	managerName[managerCount] = nameValidation(15,1);
	cout << endl;
	cout <<"\t\t\t Enter Manager User Name (4-10 characters): ";
	managerUsername[managerCount] = usernameValidation(10,4);

	cout << endl;
	cout <<"\t\t\t Enter Manager Password (4-10 characters): ";
	managerPassword[managerCount] = passwordValidation(10,4);
	cout << endl;
	cout <<"\t\t\t Enter Manager Mobile Number (11 digits): ";
	managerPhoneNumber[managerCount] = numberValidation(11,11);
```

```
	cout << endl;
	cout <<"\t\t\t Enter Manager CNIC (13 digits): ";
	managerCNIC[managerCount] = numberValidation(13,13);
	cout << endl;
	cout <<"\t\t\t Enter Manager Salary (4-7 digits): ";
	managerSalary[managerCount] = (7, 4);

	eligible = checkAccountEligibility (managerUsername, managerCount);
	return eligible;
}



string __update_managerDetailsMenu(int managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerPhoneNumber[], string
managerCNIC[])
{
	int index;
	bool check = false;
	string name;

	if (managerCount == 0)
	{
		return "No Manager to Update...";
	}

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter Username of the Manager you want to Update: ";
	getline(cin, name);
```

```
for (int x = 0; x < managerCount; x++)
{
    if (name == managerUsername[x])
    {
        index = x;
        check = true;
    }
}

if (check == false)
{
    return "No Manager exists with this Username or Password...";
}

cout << endl ;
SetConsoleTextAttribute(h, 14);
cout <<"\t Details of the Manager before Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Password"<< setw(30) << left <<"Mobile Number"<< setw(30) << left <<"CNIC"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << managerName[index] << setw(30) << left << managerUsername[index] << setw(30) << left << managerPassword[index] << setw(30) << left << managerPhoneNumber[index] << setw(30) << left << managerCNIC[index] << endl << endl;

SetConsoleTextAttribute(h, 14);
cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;
```

```
cout <<"\t\t\t Enter new Full Name: ";
managerName[index] = nameValidation(15,3);
cout << endl;
cout <<"\t\t\t Enter new User Name: ";
managerUsername[index] = usernameValidation(10,4);
cout << endl;
cout <<"\t\t\t Enter new Password: ";
managerPassword[index] = passwordValidation(10,4);
cout << endl;
cout <<"\t\t\t Enter new Mobile Number (11 digits): ";
managerPhoneNumber[index] = numberValidation(11,11);
cout << endl;
cout <<"\t\t\t Enter new CNIC (13 digits): ";
managerCNIC[index] = numberValidation(13,13);

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Details of the Manager after Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Password"<< setw(30) << left <<"Mobile Number"<< setw(30) << left <<"CNIC"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << managerName[index] << setw(30) << left << managerUsername[index] << setw(30) << left << managerPassword[index] << setw(30) << left << managerPhoneNumber[index] << setw(30) << left << managerCNIC[index] << endl << endl;
```

```
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to Continue...";
	getch();

	return "Manager updated Successfully...";
}



string __remove_managerMenu(int &managerCount, string managerUsername[], string
managerPassword[], string managerName[], string managerPhoneNumber[], string
managerCNIC[], string managerSalary[])
{
	int index;
	bool check = false;
	string name;

	if (managerCount == 0)
	{
		return "No Manager to Remove...";
	}

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter Username of the Manager you want to Remove: ";
	getline(cin, name);

	for (int x = 0; x < managerCount; x++)
	{
		if (name == managerUsername[x])
```

```
            {
                index = x;
                check = true;
            }
        }

        if (check == false)
        {
            return "No Manager exists with this Username or Password...";
        }

        for (int y = index; y < managerCount; y++)
        {
            managerUsername[y] = managerUsername[y+1];
            managerPassword[y] = managerPassword[y+1];
            managerName[y] = managerName[y+1];
            managerPhoneNumber[y] = managerPhoneNumber[y+1];
            managerCNIC[y] = managerCNIC[y+1];
            managerSalary[y] = managerSalary[y+1];
        }
        managerCount--;

        return "Manager removed Successfully...";
}
```

string __update_managerSalaryMenu(int managerCount, string managerUsername[], string managerPassword[], string managerName[], string managerSalary[])

```
{
	int index;
	bool check = false;
	string name;

	if (managerCount == 0)
	{
		return "No Manager to Update...";
	}

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter Username of the manager you want to Update: ";
	getline(cin, name);

	for (int x = 0; x < managerCount; x++)
	{
		if (name == managerUsername[x])
		{
			index = x;
			check = true;
		}
	}

	if (check == false)
	{
		return "No Manager exists with this Username or Password...";
	}
```

```
cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Salary of Manager before Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Salary"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << managerName[index] << setw(30) << left << managerUsername[index] << setw(30) << left << managerSalary[index] << endl << endl;

SetConsoleTextAttribute(h, 14);
cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

cout <<"\t\t\t Enter new Salary (4-7 digits): ";
managerSalary[index] = numberValidation(7,4);

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Salary of Manager after Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Salary"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << managerName[index] << setw(30) << left << managerUsername[index] << setw(30) << left << managerSalary[index] << endl << endl;

cout << endl << endl;
```

```
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to Continue...";
        getch();

        return "Manager updated Successfully...";
}


void viewExpensesMenu (int managerCount, string managerSalary[], int employeeCount, string
employeeSalary[])
{
        double employeeSalaries = 0;
        double managerSalaries = 0;
        double serviceCenter = 600000;
        double showroom = 700000;
        double other = 200000;
        double total;

        for (int x = 0; x < managerCount; x++)
        {
                managerSalaries += stoi(managerSalary[x]);
        }
        for (int y = 0; y < employeeCount; y++)
        {
                employeeSalaries += stoi(employeeSalary[y]);
        }
        total = managerSalaries + employeeSalaries + serviceCenter + showroom + other;
```

```
	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Details of Expences are given below... :"<< endl << endl;
	SetConsoleTextAttribute(h, 13);
	cout <<"	"<< setw(30) << left <<"Total Manager Salaries"<< setw(30) << left <<"Total Employees Salaries"<< setw(30) << left <<"Service Center Expenses"<< setw(30) << left <<"ShowRoom Expenses"<< setw(30) << left <<"Other Expanses"<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"	"<< setw(30) << left << setprecision(12) << managerSalaries << setw(30) << left << setprecision(12) << employeeSalaries << setw(30) << left << setprecision(12) << serviceCenter << setw(30) << left << setprecision(12) << showroom << setw(30) << left << setprecision(12) << other << endl << endl << endl;
	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t "<< setw(30) << left <<"Grand Total "<< setw(50) << left << setprecision(15) << total << endl;


	SetConsoleTextAttribute(h, 8);
	cout << endl << endl;
	cout <<"\t\t Press any key to Continue...";
	getch();
}



void _viewCommentsMenu (int customerCommentsCount,  string commentingCustomerName[], string commentingCustomerUsername[], string customerComments[])
{
	if (customerCommentsCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout << endl << endl;
		cout <<"\t\t No comments given by Customers yet..."<< endl;
```

```
        }
        else if (customerCommentsCount > 0)
        {
                SetConsoleTextAttribute(h, 13);
                cout <<"  "<< setw(10) << left <<"SrNo."<< setw(10) << left <<"Name"<<
setw(15) << left <<"UserName"<< setw(10) << left <<"| "<<"Comments" << endl;
                SetConsoleTextAttribute(h, 11);
                for (int x = 0; x < customerCommentsCount; x++)
                {
                        cout <<"  "<< setw(10) << left << x+1 << setw(10) << left <<
commentingCustomerName[x] << setw(15) << left << commentingCustomerUsername[x] <<
setw(10) << left <<"| "<< customerComments[x] << endl;
                }
        }
        cout << endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}



string managerMenu()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t  1) Important Announcements"<< endl << endl;
```

```
	cout <<"\t\t\t  2) Contact to Owner"<< endl << endl;
	cout <<"\t\t\t  3) Veiw Vehicles Details"<< endl << endl;
	cout <<"\t\t\t  4) Modify Vehicles Details"<< endl << endl;
	cout <<"\t\t\t  5) Veiw Associated Products Details"<< endl << endl;
	cout <<"\t\t\t  6) Modify Associated Products Details"<< endl << endl;
	cout <<"\t\t\t  7) Sold Vehicles and Products"<< endl << endl;
	cout <<"\t\t\t  8) Veiw Employees Details"<< endl << endl;
	cout <<"\t\t\t  9) Manage Employees"<< endl << endl;
	cout <<"\t\t\t 10) Scheduled Registrations for Service"<< endl << endl;
	cout <<"\t\t\t 11) Scheduled Registrations for Test Drives"<< endl << endl;
	cout <<"\t\t\t 12) Communicate Employees"<< endl << endl;
	cout <<"\t\t\t 13) View Feedbacks and Opinions"<< endl << endl;
	cout <<"\t\t\t 14) Account Settings"<< endl << endl;
	cout <<"\t\t\t 15) Sign out"<< endl << endl;
	cout <<"\t\t\t 16) Exit Application"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-16): ";
	getline(cin, option);

	return option;
}



void _showOwnerAnnouncements (string ownerNotices[], int ownerNoticesCount)
{
	if (ownerNoticesCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
```

```
		cout << endl << endl;
		cout <<"\t\t No Announcements by Owner yet."<< endl;
	}
	else if (ownerNoticesCount > 0)
	{
		SetConsoleTextAttribute(h, 14);
		cout <<"\t\t Important Announcements by Owner are: "<< endl << endl;
		SetConsoleTextAttribute(h, 11);
		for (int x = 0; x < ownerNoticesCount; x++)
		{
			cout << "  " << setw(15) << left << x+1 << ownerNotices[x] << endl;
		}
	}
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to continue...";
	getch();
}



string _contactOwnerMenu (int managerCount, string managerName[], string
managerUsername[], string temporaryUsername, string managerName2[], string
managerUsername2[], string managerMessages[], int &managerMessagesCount)
{
	int index;
	for (int x = 0; x < managerCount; x++)
	{
		if (temporaryUsername == managerUsername[x])
		{
```

```
                index = x;
            }
        }

        managerName2[managerMessagesCount] = managerName[index];
        managerUsername2[managerMessagesCount] = managerUsername[index];

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Deliver your message to Owner... "<< endl << endl;
        cout << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\tEnter your Message: ";
        getline(cin>>ws, managerMessages[managerMessagesCount]);

        managerMessagesCount++;
        return "Your message has been delivered to the Owner Successfully...";
}



void _viewVehicle_detailsMenu (int vehicleCount, string vehicleMake[], string vehicleName[],
string vehicleStock[], string vehiclePrice[])
{
        if (vehicleCount == 0)
        {
            SetConsoleTextAttribute(h, 12);
            cout << endl << endl;
            cout <<"\t\t No Vehicles Available..."<< endl;
        }
```

```
        else
        {
                SetConsoleTextAttribute(h, 13);
                cout <<" "<< setw(15) << left <<"SrNo."<< setw(35) << left <<"Make"<<
setw(35) << left <<"Name"<< setw(35) << left <<"Stock"<< setw(35) << left <<"Price"<< endl
<< endl;
                SetConsoleTextAttribute(h, 11);
                for (int x = 0; x < vehicleCount; x++)
                {
                        cout <<" "<< setw(15) << left << x + 1 << setw(35) << left <<
vehicleMake[x] << setw(35) << left << vehicleName[x] << setw(35) << left << vehicleStock[x]
<< setw(35) << left << vehiclePrice[x] << endl << endl;
                }
        }
        cout << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}



string _modifyVehiclesMenu ()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Add Vehicle"<< endl << endl;
```

```
	cout <<"\t\t\t 2) Update Vehicle"<< endl << endl;
	cout <<"\t\t\t 3) Remove Vehicle"<< endl << endl;
	cout <<"\t\t\t 4) Go Back"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-4): ";
	getline(cin, option);

	return option;
}



string __addVehicleMenu (int &vehicleCount, string vehicleMake[], string vehicleName[],
string vehicleStock[], string vehiclePrice[])
{
	string temp1, temp2;
	bool eligible = true;

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t Enter the Make of Vehicle: ";
	getline(cin, temp1);
	cout <<"\t\t Enter the Name of Vehicle: ";
	getline(cin, temp2);

	eligible = checkEligibility (vehicleName, temp2, vehicleCount);
	if (eligible == false)
	{
		return "Error!! Vehicle already exists...";
	}
```

```
        vehicleMake[vehicleCount] = temp1;
        vehicleName[vehicleCount] = temp2;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

        cout <<"\t\t Enter the Stock of Vehicle: ";
        vehicleStock[vehicleCount] = numberValidation(4, 1);
        cout << endl;
        cout <<"\t\t Enter the Price of Vehicle: ";
        vehiclePrice[vehicleCount] = numberValidation(9,5);
        cout << endl;
        vehicleCount++;
        return "New Vehicle has been added successfully...";
}


string __updateVehicleMenu (int vehicleCount, string vehicleMake[], string vehicleName[],
string vehicleStock[], string vehiclePrice[])
{
        int index;
        bool check = false;
        string make, name;

        if (vehicleCount == 0)
        {
                return "No vehicle to Update...";
```

```
}

SetConsoleTextAttribute(h, 11);
cout <<"\t\t Enter the Make of the Vehicle you Want to Update: ";
getline(cin, make);
cout <<"\t\t Enter the Name of the Vehicle you Want to Update: ";
getline(cin, name);

for (int x = 0; x < vehicleCount; x++)
{
	if (make == vehicleMake[x] && name == vehicleName[x])
	{
		index = x;
		check = true;
	}
}

if (check == false)
{
	return "No vehicle exists with this Name or Make...";
}

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Details of Vehicle before Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Make"<< setw(30) << left <<"Name"<< setw(30) << left <<"Stock"<< setw(30) << left <<"Price"<< endl << endl;
```

```
SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << vehicleMake[index] << setw(30) << left << vehicleName[index] << setw(30) << left << vehicleStock[index] << setw(30) << left << vehiclePrice[index] << endl << endl;

SetConsoleTextAttribute(h, 14);
cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

cout <<"\t\t\t Enter new Stock: ";
vehicleStock[index] = numberValidation(4, 1);
cout << endl;
cout <<"\t\t\t Enter new Price: ";
vehiclePrice[index] = numberValidation(9,5);

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Details of Vehicle after Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Make"<< setw(30) << left <<"Name"<< setw(30) << left <<"Stock"<< setw(30) << left <<"Price"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << vehicleMake[index] << setw(30) << left << vehicleName[index] << setw(30) << left << vehicleStock[index] << setw(30) << left << vehiclePrice[index] << endl << endl;

cout << endl << endl;
SetConsoleTextAttribute(h, 8);
cout <<"\t\t Press any key to Continue...";
```

```
        getch();

        return "Vehicle updated Successfully...";
}



string __removeVehicleMenu (int &vehicleCount, string vehicleMake[], string vehicleName[],
string vehicleStock[], string vehiclePrice[])
{
        int index;
        bool check = false;
        string make, name;

        if (vehicleCount == 0)
        {
                return "No vehicle to Remove...";
        }

        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t Enter the Make of the Vehicle you Want to Remove: ";
        getline(cin, make);
        cout <<"\t\t Enter the Name of the Vehicle you Want to Remove: ";
        getline(cin, name);

        for (int x = 0; x < vehicleCount; x++)
        {
                if (make == vehicleMake[x] && name == vehicleName[x])
                {
```

```
                    index = x;
                    check = true;
            }
        }

        if (check == false)
        {
            return "No vehicle exists with this Name or Model...";
        }

        for (int y = index; y < vehicleCount; y++)
        {
            vehicleMake[y] = vehicleMake[y+1];
            vehicleName[y] = vehicleName[y+1];
            vehicleStock[y] = vehicleStock[y+1];
            vehiclePrice[y] = vehiclePrice[y+1];
        }
        vehicleCount--;

        return "Vehicle removed Successfully...";
}



void _viewProduct_detailsMenu (int productCount, string productName[], string productStock[],
string productPrice[])
{
        if (productCount == 0)
        {
```

```
                SetConsoleTextAttribute(h, 12);
                cout << endl << endl;
                cout <<"\t\t No Vehicles Available..."<< endl;
        }
        else
        {
                SetConsoleTextAttribute(h, 13);
                cout <<"  "<< setw(15) << left <<"SrNo."<< setw(30) << left <<"Name"<<
setw(30) << left <<"Stock"<< setw(30) << left <<"Price"<< endl << endl;
                SetConsoleTextAttribute(h, 11);
                for (int x = 0; x < productCount; x++)
                {
                        cout <<"  "<< setw(15) << left << x + 1 << setw(30) << left <<
productName[x] << setw(30) << left << productStock[x] << setw(30) << left << productPrice[x]
<< endl << endl;
                }
        }
        cout << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}



string _modifyProductMenu ()
{
        string option;

        SetConsoleTextAttribute(h, 14);
```

```
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Add Product"<< endl << endl;
        cout <<"\t\t\t 2) Update Product"<< endl << endl;
        cout <<"\t\t\t 3) Remove Product"<< endl << endl;
        cout <<"\t\t\t 4) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-4): ";
        getline(cin, option);

        return option;
}



string __addProductMenu (int &productCount, string productName[], string productStock[],
string productPrice[])
{
        int index = productCount;
        string temp;
        bool eligible = true;

        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t Enter the Name of Product: ";
        getline(cin, temp);

        eligible = checkEligibility (productName, temp, productCount);
        if (eligible == false)
        {
```

```
            return "Error!! Product already exists...";
        }

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

        productName[index] = temp;
        cout <<"\t\t Enter the Stock of Product: ";
        productStock[index] = numberValidation(4,1);
        cout << endl;
        cout <<"\t\t Enter the Price of Product: ";
        productPrice[index] = numberValidation(9,2);
        cout << endl;
        productCount++;
        return "New Product has been added successfully...";
}


string __updateProductMenu (int productCount, string productName[], string productStock[],
string productPrice[])
{
        int index;
        bool check = false;
        string name;

        if (productCount == 0)
        {
            return "No product to Update...";
```

```
}

SetConsoleTextAttribute(h, 11);
cout <<"\t\t Enter the Name of the Product you Want to Update: ";
getline(cin, name);

for (int x = 0; x < productCount; x++)
{
    if (name == productName[x])
    {
        index = x;
        check = true;
    }
}

if (check == false)
{
    return "No product exists with this Name...";
}

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Details of Product before Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"   "<< setw(30) << left <<"Name"<< setw(30) << left <<"Stock"<< setw(30) << left <<"Price"<< endl << endl;

SetConsoleTextAttribute(h, 11);
```

```
    cout <<"   "<< setw(30) << left << productName[index] << setw(30) << left << productStock[index] << setw(30) << left << productPrice[index] << endl << endl;

    SetConsoleTextAttribute(h, 14);
    cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

    cout <<"\t\t\t Enter new Stock: ";
    productStock[index] = numberValidation(4,1);
    cout << endl;
    cout <<"\t\t\t Enter new Price: ";
    productPrice[index] = numberValidation(9,2);

    cout << endl << endl;
    SetConsoleTextAttribute(h, 14);
    cout <<"\t Details of Product after Update: "<< endl << endl;
    SetConsoleTextAttribute(h, 13);
    cout <<"   "<< setw(30) << left <<"Name"<< setw(30) << left <<"Stock"<< setw(30) << left <<"Price"<< endl << endl;

    SetConsoleTextAttribute(h, 11);
    cout <<"   "<< setw(30) << left << productName[index] << setw(30) << left << productStock[index] << setw(30) << left << productPrice[index] << endl << endl;

    cout << endl << endl;
    SetConsoleTextAttribute(h, 8);
    cout <<"\t\t Press any key to Continue...";
    getch();

    return "Product updated Successfully...";
```

```
}



string __removeProductMenu (int &productCount, string productName[], string productStock[],
string productPrice[])
{
	int index;
	bool check = false;
	string name;

	if (productCount == 0)
	{
		return "No product to Remove...";
	}

	SetConsoleTextAttribute(h, 11);
	cout <<"\t Enter the Name of the Product you Want to Remove: ";
	getline(cin, name);

	for (int x = 0; x < productCount; x++)
	{
		if (name == productName[x])
		{
			index = x;
			check = true;
		}
	}
```

```
        if (check == false)
        {
                return "No product exists with this Name...";
        }

        for (int y = index; y < productCount; y++)
        {
                productName[y] = productName[y+1];
                productStock[y] = productStock[y+1];
                productPrice[y] = productPrice[y+1];
        }
        productCount--;

        return "Product removed Successfully...";
}


string soldItemsMenu ()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Select From the Following... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Sold Vehicles"<< endl << endl;
        cout <<"\t\t\t 2) Sold Products"<< endl << endl;
        cout <<"\t\t\t 3) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
```

```
	cout <<"\t\t Enter Your Option (1-3): ";
	getline(cin, option);

	return option;
}



void soldVehiclesMenu (int buyVehicleCustomerCount, string buyVehicleCustomerName[],
string buyVehicleCustomerPhoneNumber[], string buyVehicleName[], string buyVehicleMake[],
string buyVehiclePrice[], string buyVehiclePayment[])
{
	if (buyVehicleCustomerCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout <<"\t\t No Vehicle has been purchased till now..."<< endl;
	}

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Vehicles Purchased :"<< buyVehicleCustomerCount << endl << endl;
	SetConsoleTextAttribute(h, 13);
	cout <<" "<< setw(10) << left <<"SrNo."<< setw(20) << left <<"Name"<< setw(20) <<
left <<"Phone Number"<< setw(20) << left <<"Vehicle Make"<< setw(20) << left <<"Vehicle
Name"<< setw(20) << left <<"Vehicle Price"<< setw(20) << left <<"Payment Method"<< endl
<< endl;
	SetConsoleTextAttribute(h, 11);
	for (int x = 0; x < buyVehicleCustomerCount; x++)
	{
		cout <<" "<< setw(10) << left << x + 1 << setw(20) << left <<
buyVehicleCustomerName[x] << setw(20) << left << buyVehicleCustomerPhoneNumber[x] <<
setw(20) << left << buyVehicleMake[x] << setw(20) << left << buyVehicleName[x] <<
```

```
setw(20) << left << buyVehiclePrice[x] << setw(20) << left << buyVehiclePayment[x] << endl
<< endl;
        }

        SetConsoleTextAttribute(h, 8);
        cout << endl << endl;
        cout <<"\t\t Press any key to Continue...";
        getch();
}


void soldProductsMenu (int buyProductCustomerCount, string buyProductCustomerName[],
string buyProductCustomerPhoneNumber[], string buyProductName[], string buyProductPrice[],
string buyProductPayment[])
{
        if (buyProductCustomerCount == 0)
        {
                SetConsoleTextAttribute(h, 12);
                cout <<"\t\t No Product has been purchased till now..."<< endl;
        }

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Products Purchased :"<< buyProductCustomerCount << endl << endl;
        SetConsoleTextAttribute(h, 13);
        cout <<" "<< setw(10) << left <<"SrNo."<< setw(20) << left <<"Name"<< setw(20) <<
left <<"Phone Number"<< setw(20) << left <<"Product Name"<< setw(20) << left <<"Product
Price"<< setw(20) << left <<"Payment Method"<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        for (int x = 0; x < buyProductCustomerCount; x++)
        {
```

```
		cout <<" "<< setw(10) << left << x + 1 << setw(20) << left << buyProductCustomerName[x] << setw(20) << left << buyProductCustomerPhoneNumber[x] << setw(20) << left << buyProductName[x] << setw(20) << left << buyProductPrice[x] << setw(20) << left << buyProductPayment[x] << endl << endl;
	}

	SetConsoleTextAttribute(h, 8);
	cout << endl << endl;
	cout <<"\t\t Press any key to Continue...";
	getch();
}


void _viewEmployeesMenu (int employeeCount, string employeeName[], string employeeUsername[], string employeePhoneNumber[], string employeeCNIC[], string employeeSalary[])
{
	if (employeeCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout << endl << endl;
		cout <<"\t\t No Employees Added Yet."<< endl;
	}
	else
	{
		SetConsoleTextAttribute(h, 13);
		cout <<" "<< setw(15) << left <<"SrNo."<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Mobile Number"<< setw(30) << left <<"CNIC"<< setw(30) << left <<"Salary"<< endl << endl;
		SetConsoleTextAttribute(h, 11);
```

```
		for (int x = 0; x < employeeCount; x++)
		{
			cout <<" "<< setw(15) << left << x + 1 << setw(30) << left << 
employeeName[x] << setw(30) << left << employeeUsername[x] << setw(30) << left << 
employeePhoneNumber[x] << setw(30) << left << employeeCNIC[x] << setw(30) << left << 
employeeSalary[x] << endl << endl;
		}
	}
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to continue...";
	getch();
}



string _manage_employeeMenu()
{
	string option;

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Choose you want to do.... "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t 1) Add an Employee"<< endl << endl;
	cout <<"\t\t\t 2) Update Employee Details"<< endl << endl;
	cout <<"\t\t\t 3) Remove an Employee"<< endl << endl;
	cout <<"\t\t\t 4) Update Employee Salary"<< endl << endl;
	cout <<"\t\t\t 5) Go Back"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-5): ";
```

```
        getline(cin, option);

        return option;
}


bool __add_employeeMenu(int employeeCount, string employeeUsername[], string
employeePassword[], string employeeName[], string employeePhoneNumber[], string
employeeCNIC[], string employeeSalary[])
{
        bool eligible;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t Enter Employee Full Name: ";
        employeeName[employeeCount] = nameValidation(15,3);
        cout << endl;
        cout <<"\t\t\t Enter Employee User Name: ";
        employeeUsername[employeeCount] = usernameValidation(10,4);

        cout << endl;
        cout <<"\t\t\t Enter Employee Password: ";
        employeePassword[employeeCount] = passwordValidation(10,4);
        cout << endl;
        cout <<"\t\t\t Enter Employee Mobile Number (11 digits): ";
        employeePhoneNumber[employeeCount] = numberValidation(11,11);
        cout << endl;
```

```
        cout <<"\t\t\t Enter Employee CNIC: ";
        employeeCNIC[employeeCount] = numberValidation(13,13);
        cout << endl;
        cout <<"\t\t\t Enter Employee Salary: ";
        employeeSalary[employeeCount] = numberValidation(7,4);

        eligible = checkAccountEligibility (employeeUsername, employeeCount);
        return eligible;
}



string __update_employeeDetailsMenu(int employeeCount, string employeeUsername[], string
employeePassword[], string employeeName[], string employeePhoneNumber[], string
employeeCNIC[])
{
        int index;
        bool check = false;
        string name;

        if (employeeCount == 0)
        {
                return "No Employee to Remove...";
        }

        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t Enter Username of the Employee you want to Update: ";
        getline(cin, name);

        for (int x = 0; x < employeeCount; x++)
```

```
{
        if (name == employeeUsername[x])
        {
                index = x;
                check = true;
        }
}

if (check == false)
{
        return "No Employee exists with this Username or Password...";
}

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

SetConsoleTextAttribute(h, 14);
cout <<"\t Details of the Employee before Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<" "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Password"<< setw(30) << left <<"Mobile Number"<< setw(30) << left <<"CNIC"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<" "<< setw(30) << left << employeeName[index] << setw(30) << left << employeeUsername[index] << setw(30) << left << employeePassword[index] << setw(30) << left << employeePhoneNumber[index] << setw(30) << left << employeeCNIC[index] << endl << endl;
```

```
cout <<"\t\t\t Enter new Full Name: ";
employeeName[index] = nameValidation(15,3);
cout << endl;
cout <<"\t\t\t Enter new User Name: ";
employeeUsername[index] = usernameValidation(10,4);

cout << endl;
cout <<"\t\t\t Enter new Password: ";
employeePassword[index] = passwordValidation(10,4);
cout << endl;
cout <<"\t\t\t Enter new Mobile Number: ";
employeePhoneNumber[index] = numberValidation(11,11);
cout << endl;
cout <<"\t\t\t Enter new CNIC: ";
employeeCNIC[index] = numberValidation(13,13);

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Details of the Employee after Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<" "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Password"<< setw(30) << left <<"Mobile Number"<< setw(30) << left <<"CNIC"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<" "<< setw(30) << left << employeeName[index] << setw(30) << left << employeeUsername[index] << setw(30) << left << employeePassword[index] << setw(30) << left << employeePhoneNumber[index] << setw(30) << left << employeeCNIC[index] << endl << endl;
```

```
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to Continue...";
	getch();

	return "Employee updated Successfully...";
}



string __remove_employeeMenu(int &employeeCount, string employeeUsername[], string
employeePassword[], string employeeName[], string employeePhoneNumber[], string
employeeCNIC[], string employeeSalary[])
{
	int index;
	bool check = false;
	string name;

	if (employeeCount == 0)
	{
		return "No Employee to Remove...";
	}

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter Username of the Employee you want to Remove: ";
	getline(cin, name);

	for (int x = 0; x < employeeCount; x++)
	{
		if (name == employeeUsername[x])
```

```
            {
                index = x;
                check = true;
            }
        }

        if (check == false)
        {
            return "No Employee exists with this Username or Password...";
        }

        for (int y = index; y < employeeCount; y++)
        {
            employeeUsername[y] = employeeUsername[y+1];
            employeePassword[y] = employeePassword[y+1];
            employeeName[y] = employeeName[y+1];
            employeePhoneNumber[y] = employeePhoneNumber[y+1];
            employeeCNIC[y] = employeeCNIC[y+1];
            employeeSalary[y] = employeeSalary[y+1];
        }
        employeeCount--;

        return "Employee removed Successfully...";
}
```

string __update_employeeSalaryMenu(int employeeCount, string employeeUsername[], string employeePassword[], string employeeName[], string employeeSalary[])

```
{
	int index;
	bool check = false;
	string name;

	if (employeeCount == 0)
	{
		return "No Employee to Update...";
	}

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter Username of the Employee you want to Update: ";
	getline(cin, name);

	for (int x = 0; x < employeeCount; x++)
	{
		if (name == employeeUsername[x])
		{
			index = x;
			check = true;
		}
	}

	if (check == false)
	{
		return "No Employee exists with this Username or Password...";
	}
```

```
cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Salary of the Employee before Update: "<< endl << endl;
SetConsoleTextAttribute(h, 13);
cout <<"  "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Salary"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << employeeName[index] << setw(30) << left << employeeUsername[index] << setw(30) << left << employeeSalary[index] << endl << endl;

cout <<"\t\t\t Enter new Salary: ";
employeeSalary[index] = numberValidation(7,4);

cout << endl << endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t Salary of the Employee after Update: "<< endl << endl;
cout <<"  "<< setw(30) << left <<"Name"<< setw(30) << left <<"Username"<< setw(30) << left <<"Salary"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << employeeName[index] << setw(30) << left << employeeUsername[index] << setw(30) << left << employeeSalary[index] << endl << endl;

cout << endl << endl;
SetConsoleTextAttribute(h, 8);
cout <<"\t\t Press any key to Continue...";
getch();
```

```
	return "Employee updated Successfully...";
}



void _serviceDetailsMenu (int customer4ServiceCount,  string customer4ServiceName[], string
customer4ServiceUsername[], string serviceVehicleMake[], string serviceVehicleName[])
{
	if (customer4ServiceCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout <<"\t No Registrations Yet..."<< endl << endl;
	}
	else
	{
		SetConsoleTextAttribute(h, 14);
		cout <<"\t Details of the Customer Registered for Vehicle Service: "<< endl <<
endl;
		SetConsoleTextAttribute(h, 13);
		cout <<" "<< setw(15) << left <<"SrNo."<< setw(30) << left <<"Customer
Name"<< setw(30) << left <<"Customer Username"<< setw(30) << left <<"Vehicle Make"<<
setw(30) << left <<"Vehicle Name"<< endl << endl;
		SetConsoleTextAttribute(h, 11);
		for (int x = 0; x < customer4ServiceCount; x++)
		{
			cout <<" "<< setw(15) << left << x + 1 << setw(30) << left <<
customer4ServiceName[x] << setw(30) << left << customer4ServiceUsername[x] << setw(30)
<< left << serviceVehicleMake[x] << setw(30) << left << serviceVehicleName[x]  << endl;
		}
	}
```

```
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to continue...";
	getch();
}


void _testDriveDetailsMenu (int customer4DriveCount,  string customer4DriveName[], string
customer4DriveUsername[], string customer4DriveVehicle[])
{
	if (customer4DriveCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout <<"\t No Registrations Yet..."<< endl << endl;
	}
	else
	{
		SetConsoleTextAttribute(h, 14);
		cout << endl;
		cout <<"\t Details of the Customer Registered for Testing Drive: "<< endl <<
endl;
		SetConsoleTextAttribute(h, 13);
		cout <<" "<< setw(15) << left <<"SrNo."<< setw(30) << left <<"Customer
Name"<< setw(30) << left <<"Customer Username"<< setw(40) << left <<"Vehicle Chosen" <<
endl;
		SetConsoleTextAttribute(h, 11);
		for (int x = 0; x < customer4DriveCount; x++)
		{
```

```
                    cout <<" "<< setw(15) << left << x + 1 << setw(30) << left << customer4DriveName[x] << setw(30) << left << customer4DriveUsername[x] << setw(40) << left << customer4DriveVehicle[x]  << endl;
                }
        }

        cout << endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}


string _communicateEmployeesMenu ()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Announce Important Message"<< endl << endl;
        cout <<"\t\t\t 2) Message received by Employees"<< endl << endl;
        cout <<"\t\t\t 3) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-3): ";
        getline(cin, option);

        return option;
}
```

```
string __managerAnnouncements (string managerNotices[], int &managerNoticesCount)
{
	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Announce Important Message to Employees: "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Your Announcement: ";
	getline(cin, managerNotices[managerNoticesCount]);
	managerNoticesCount++;

	return "Your Message has been Delivered Successfully...";
}


void __receiveEmployeeMessagesMenu (string employeeName2[], string
employeeUsername2[], string employeeMessages[], int employeeMessagesCount)
{
	if (employeeMessagesCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout << endl << endl;
		cout <<"\t\t No messages received by any Employee yet."<< endl;
	}
	else if (employeeMessagesCount > 0)
	{
		SetConsoleTextAttribute(h, 13);
		cout <<"  "<< setw(7) << left <<"SrNo."<< setw(10) << left <<"Name"<<
setw(15) << left <<"UserName"<< setw(10) << left <<"Message" << endl;
```

```
                SetConsoleTextAttribute(h, 11);
                for (int x = 0; x < employeeMessagesCount; x++)
                {
                        cout <<"  "<< setw(7) << left << x+1 << setw(10) << left <<
employeeName2[x] << setw(15) << left << employeeUsername2[x] << setw(10) << left
<<employeeMessages[x] << endl;
                }
        }
        cout << endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}


string employeeMenu()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to do.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Important Announcements"<< endl << endl;
        cout <<"\t\t\t 2) Contact to Manager"<< endl << endl;
        cout <<"\t\t\t 3) View Vehicle Details"<< endl << endl;
        cout <<"\t\t\t 4) View Accociated Products Details"<< endl << endl;
        cout <<"\t\t\t 5) Sold Vehicles and Products"<< endl << endl;
        cout <<"\t\t\t 6) Scheduled Registrations for Service"<< endl << endl;
        cout <<"\t\t\t 7) Scheduled Registrations for Test Drives"<< endl << endl;
```

```
	cout <<"\t\t\t 8) Account Settings"<< endl << endl;
	cout <<"\t\t\t 9) Sign out"<< endl << endl;
	cout <<"\t\t\t10) Exit Application"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-10): ";
	getline(cin, option);

	return option;
}



void _showManagerAnnouncements (string managerNotices[], int managerNoticesCount)
{
	if (managerNoticesCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout << endl << endl;
		cout <<"\t\t No Announcements by Manager yet."<< endl;
	}
	else if (managerNoticesCount > 0)
	{
		SetConsoleTextAttribute(h, 14);
		cout <<"\t Important Announcements by Manager are: "<< endl << endl;
		SetConsoleTextAttribute(h, 11);
		for (int x = 0; x < managerNoticesCount; x++)
		{
			cout <<"  "<< setw(15)<< left << x+1 << managerNotices[x] << endl;
		}
```

```
        }
        cout << endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}


string _contactManagerMenu (int employeeCount, string employeeName[], string
employeeUsername[], string temporaryUsername, string employeeName2[], string
employeeUsername2[], string employeeMessages[], int &employeeMessagesCount)
{
        int index;
        for (int x = 0; x < employeeCount; x++)
        {
                if (temporaryUsername == employeeUsername[x])
                {
                        index = x;
                }
        }

        employeeName2[employeeMessagesCount] = employeeName[index];
        employeeUsername2[employeeMessagesCount] = employeeUsername[index];

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Deliver your message to Manager... "<< endl << endl;

        SetConsoleTextAttribute(h, 11);
        cout << endl;
```

```
        cout <<"\t\t\tEnter your Message: ";
        getline(cin>>ws, employeeMessages[employeeMessagesCount]);

        employeeMessagesCount++;
        return "Your message has been delivered to the Manager Successfully...";
}



string accountSettingMenu ()
{
   string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to change.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Name"<< endl << endl;
        cout <<"\t\t\t 2) Username"<< endl << endl;
        cout <<"\t\t\t 3) Password"<< endl << endl;
        cout <<"\t\t\t 4) Mobile Number"<< endl << endl;
        cout <<"\t\t\t 5) CNIC"<< endl << endl;
        cout <<"\t\t\t 6) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-6): ";
        getline(cin, option);

        return option;
}
```

```
string changeProfile (int count, string username[], string temporaryUsername, string data[],
string message)
{
	int index;
	for (int x = 0; x < count; x++)
	{
		if (temporaryUsername == username[x])
		{
			index = x;
		}
	}

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;

	SetConsoleTextAttribute(h, 13);
	cout <<"\t\t Your old "<< message <<": ";
	SetConsoleTextAttribute(h, 14);
	cout << data[index] << endl << endl << endl;

	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t  Enter Your new "<< message <<": "<< endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t
*************************************************************************"<<
endl<< "\t\t  ";
	SetConsoleTextAttribute(h, 11);
	data[index] = usernameValidation(10,3);
```

```
	cout << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t
**********************************************************************"<<
endl << endl;

	SetConsoleTextAttribute(h, 13);
	cout <<"\t\t Your New "<< message <<": ";
	SetConsoleTextAttribute(h, 14);
	cout << data[index] << endl << endl << endl;

	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to Continue...";
	getch();

	return "Your "+ message +" updated Successfully...";
}



string changeUsername (int count, string username[], string temporaryUsername, string message)
{
	int index;
	string temp;
	bool check = true;
	for (int x = 0; x < count; x++)
	{
		if (temporaryUsername == username[x])
		{
			index = x;
```

```
            }
        }

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Press Tab Key to Enter...(4-10 characters)"<< endl << endl << endl;

        SetConsoleTextAttribute(h, 13);
        cout <<"\t\t Your old "<< message <<": ";
        SetConsoleTextAttribute(h, 14);
        cout << username[index] << endl << endl << endl;
        username[index] = "-";

        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t  Enter Your new "<< message <<": "<< endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t
**********************************************************************"<<
endl<< "\t\t  ";
        SetConsoleTextAttribute(h, 11);
        temp = usernameValidation(10,4);
        cout << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t
**********************************************************************"<<
endl << endl;

        for (int x = 0; x < count; x++)
        {
            if (temp == username[x])
```

```
        {
                check = false;
        }
}

if (check == false)
{
        username[index] = temporaryUsername;
        SetConsoleTextAttribute(h, 12);
        cout <<"\t\t Username already exists!!"<< endl << endl;

        cout << endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to Continue...";
        getch();
        return "Please try again!!";
}
username[index] = temp;
SetConsoleTextAttribute(h, 13);
cout <<"\t\t Your New "<< message <<": ";
SetConsoleTextAttribute(h, 14);
cout << username[index] << endl << endl << endl;

cout << endl << endl;
SetConsoleTextAttribute(h, 8);
cout <<"\t\t Press any key to Continue...";
getch();
```

```
	return "Your "+ message +" updated Successfully...";
}



string customerMenu()
{
	string option;

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Choose you want to do.... "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t 1) View Vehicles"<< endl << endl;
	cout <<"\t\t\t 2) Buy Vehicle"<< endl << endl;
	cout <<"\t\t\t 3) Veiw Accessories and Products"<< endl << endl;
	cout <<"\t\t\t 4) Buy Associated Products"<< endl << endl;
	cout <<"\t\t\t 5) Register your Car for Service"<< endl << endl;
	cout <<"\t\t\t 6) Register for Test Drive"<< endl << endl;
	cout <<"\t\t\t 7) Add comments or Suggestions"<< endl << endl;
	cout <<"\t\t\t 8) Account Settings"<< endl << endl;
	cout <<"\t\t\t 9) Sign out"<< endl << endl;
	cout <<"\t\t\t10) Exit Application"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-10): ";
	getline(cin, option);

	return option;
}
```

```
void _customerViewVehiclesMenu (int vehicleCount, string vehicleMake[], string
vehicleName[], string vehiclePrice[])
{
	if (vehicleCount == 0)
	{
		SetConsoleTextAttribute(h, 12);
		cout << endl << endl;
		cout <<"\t\t No Vehicles Available..."<< endl;
	}
	else
	{
		SetConsoleTextAttribute(h, 13);
		cout <<" "<< setw(15) << left <<"SrNo."<< setw(35) << left <<"Make"<<
setw(35) << left <<"Name"<< setw(35) << left <<"Price"<< endl << endl;
		SetConsoleTextAttribute(h, 11);
		for (int x = 0; x < vehicleCount; x++)
		{
			cout <<" "<< setw(15) << left << x + 1 << setw(35) << left <<
vehicleMake[x] << setw(35) << left << vehicleName[x] << setw(35) << left << vehiclePrice[x]
<< endl << endl;
		}
	}
	cout << endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to continue...";
	getch();
}
```

```
string buyProductMenu (string message)
{
	string option;

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Follow Step by Step Procedure to Buy Your Desired "<< message <<"... "<<
endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t 1) Choose "<< message << endl << endl;
	cout <<"\t\t\t 2) Choose Payment Method"<< endl << endl;
	cout <<"\t\t\t 3) Print Bill"<< endl << endl;
	cout <<"\t\t\t 4) Go Back"<< endl << endl;
	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Enter Your Option (1-4): ";
	getline(cin, option);

	return option;
}


string chooseVehicleMenu (int customerCount, string customerUsername[], string
temporaryUsername, string customerName[], string customerPhoneNumber[], int vehicleCount,
string vehicleMake[], string vehicleName[], string vehiclePrice[], int buyVehicleCustomerCount,
string buyVehicleCustomerName[], string buyVehicleCustomerPhoneNumber[], string
buyVehicleName[], string buyVehicleMake[], string buyVehiclePrice[])
{
	int index1, index2;
	bool checkVehicle = false;
	bool checkCustomer = false;
```

```
char ch;
string name;

if (vehicleCount == 0)
{
	return "No vehicle to buy...";
}

for (int i = 0; i < vehicleCount; i++)
{
	if (buyVehicleName[buyVehicleCustomerCount] == vehicleName[i])
	{
		index2 = i;
		checkCustomer = true;
	}
}

if (checkCustomer == true)
{
	SetConsoleTextAttribute(h, 14);
	cout <<"\t You have already Selected this Vehicle: ";
	SetConsoleTextAttribute(h, 13);
	cout <<"  "<< setw(30) << left <<"Make"<< setw(30) << left <<"Name"<<
setw(30) << left <<"Price"<< endl << endl;

	SetConsoleTextAttribute(h, 11);
	cout <<"  "<< setw(30) << left << vehicleMake[index2] << setw(30) << left <<
vehicleName[index2] << setw(30) << left << vehiclePrice[index2] << endl << endl<< endl;
```

```
        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t If you are satisfied with this Vehicle Press ";
        SetConsoleTextAttribute(h, 15);
        cout <<"Backspace to Return,"<< endl ;
        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t If you want to change the Vehicle Press ";
        SetConsoleTextAttribute(h, 15);
        cout <<"Enter or Space to Change: ";
        while(ch = getch())
        {
            if (ch == '\b')
            {
                cout << endl;
                return "Choose the Payment Method Now...";
            }
            else if (isspace(ch))
            {
                cout << endl << endl;
                goto loop;
            }
        }
    }

    loop:
    for (int x = 0; x < customerCount; x++)
    {
        if (temporaryUsername == customerUsername[x])
        {
```

```
                index1 = x;
        }
}

SetConsoleTextAttribute(h, 14);
cout <<"\t Enter the Name of the Vehicle you want to Buy: ";
getline (cin, name);

for (int x = 0; x < vehicleCount; x++)
{
        if (name == vehicleName[x])
        {
                index2 = x;
                checkVehicle = true;
        }
}

if (checkVehicle == false)
{
        return "No vehicle exists with this Name...";
}
else if (checkVehicle == true)
{
        cout << endl;
        SetConsoleTextAttribute(h, 14);
        cout <<"\t The Vehicle available with this Name is: "<< endl << endl;
        SetConsoleTextAttribute(h, 13);
```

```
cout <<"  "<< setw(30) << left <<"Make"<< setw(30) << left <<"Name"<<
setw(30) << left <<"Price"<< endl << endl;

SetConsoleTextAttribute(h, 11);
cout <<"  "<< setw(30) << left << vehicleMake[index2] << setw(30) << left <<
vehicleName[index2] << setw(30) << left << vehiclePrice[index2] << endl << endl;

SetConsoleTextAttribute(h, 14);
cout <<"\t\t If you want to buy above Vehicle Press ";
SetConsoleTextAttribute(h, 15);
cout <<"Enter or Space to Proceed,"<< endl ;
SetConsoleTextAttribute(h, 14);
cout <<"\t\t Press ";
SetConsoleTextAttribute(h, 15);
cout <<"Backspace key to Go Back: ";

while (ch = getch())
{
    if (ch == '\b')
    {
        break;
    }
    else if (isspace(ch))
    {
        buyVehicleCustomerName[buyVehicleCustomerCount] =
customerName[index1];
        buyVehicleCustomerPhoneNumber[buyVehicleCustomerCount] =
customerPhoneNumber[index1];
        buyVehicleMake[buyVehicleCustomerCount] =
vehicleMake[index2];
```

```
                    buyVehicleName[buyVehicleCustomerCount] =
vehicleName[index2];
                    buyVehiclePrice[buyVehicleCustomerCount] =
vehiclePrice[index2];
                    cout << endl;
                    return "Vehicle Selected Successfully. Please Select the payment
method now!!";
                }
            }
        }
}



string paymentMethodMenu1 ()
{
        string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Select Full Payment method... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Immediate Cash Payment"<< endl << endl;
        cout <<"\t\t\t 2) Payment by Credit Card"<< endl << endl;
        cout <<"\t\t\t 3) Transfer Amount by Bank"<< endl << endl;
        cout <<"\t\t\t 4) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-4): ";
        getline(cin, option);

        return option;
```

```
}



string paymentMethodMenu2 (int count, string productName[], int buyCustomerCount, string
buyProductName[], string buyProductPayment[], string message1, string message2)
{
	bool checkCustomer = false;
	for (int i = 0; i < count; i++)
	{
		if (buyProductName[buyCustomerCount] == productName[i])
		{
			checkCustomer = true;
		}
	}
	if (checkCustomer == true)
	{
		buyProductPayment[buyCustomerCount] = message1;
		SetConsoleTextAttribute(h, 14);
		cout <<"\t\t Payment Method has been set to "<< message1 <<" Successfully!"<<
endl << endl;


		cout << endl << endl;
		SetConsoleTextAttribute(h, 8);
		cout <<"\t\t Press any key to Continue...";
		getch();
		return "Submit the Bill to Proceed!";
	}
	else if (checkCustomer == false)
	{
```

```
                return "Please Choose the "+ message2 +" First...";
        }
}



string vehicleBillMenu (int vehicleCount, string vehicleName[], string vehicleStock[], int
&buyVehicleCustomerCount, string buyVehicleCustomerName[], string
buyVehicleCustomerPhoneNumber[], string buyVehicleName[], string buyVehicleMake[], string
buyVehiclePrice[], string buyVehiclePayment[])
{
        int index, temp;
        bool checkCustomer = false;
        bool checkPayment = false;
    char wall = 219, ch;
    int i = 33, j = 18;


        for (int i = 0; i < vehicleCount; i++)
        {
                if (buyVehicleName[buyVehicleCustomerCount] == vehicleName[i])
                {
                        index = i;
                        checkCustomer = true;
                }
        }


        if (buyVehiclePayment[buyVehicleCustomerCount] == "Immediate Cash Payment" ||
buyVehiclePayment[buyVehicleCustomerCount] == "Payment by Credit Card" ||
buyVehiclePayment[buyVehicleCustomerCount] == "Payment By Bank")
        {
                checkPayment = true;
```

```
        }
        if (checkCustomer == false)
        {
                return "Please Choose the Vehicle First...";
        }
        if (checkPayment == false)
        {
                return "Please Choose the Payment Method First...";
        }
    char bill[26][100] =
    {

"wwww
```

```
    "ww ------------------------------------------------------------------------------------------ ww",
    "ww VEHICLE NAME :                                                                          ww",
    "ww ------------------------------------------------------------------------------------------ ww",
    "ww PAYMENT METHOD :                                                                        ww",
    "ww ------------------------------------------------------------------------------------------ ww",
    "ww                                                                                         ww",
    "ww                                                                                         ww",
    "ww ------------------------------------------------------------------------------------------ ww",
    "ww                                                RUPEES :                                 ww",
    "ww ------------------------------------------------------------------------------------------ ww",
    "ww                                                                                         ww",

"wwww
```

```
    else if (bill[x][y] == '-')
    {
        SetConsoleTextAttribute(h, 14);
        cout << bill[x][y];
    }
    else
    {
        SetConsoleTextAttribute(h, 11);
        cout << bill[x][y];
    }
}
cout << endl;
j++;
}

SetConsoleTextAttribute(h, 10);
gotoxy(44, 24);
        cout << buyVehicleCustomerName[buyVehicleCustomerCount];
gotoxy(53, 26);
        cout << buyVehicleCustomerPhoneNumber[buyVehicleCustomerCount];
gotoxy(53, 31);
        cout << buyVehicleName[buyVehicleCustomerCount];
gotoxy(52, 33);
        cout << buyVehicleMake[buyVehicleCustomerCount];
gotoxy(54, 35);
        cout << buyVehiclePayment[buyVehicleCustomerCount];
gotoxy(106, 40);
        cout << buyVehiclePrice[buyVehicleCustomerCount];
```

```
cout << endl << endl << endl << endl<< endl;
SetConsoleTextAttribute(h, 14);
cout <<"\t\t If you want to buy above Vehicle Press ";
SetConsoleTextAttribute(h, 15);
cout <<"Enter or Space to Proceed,"<< endl ;
SetConsoleTextAttribute(h, 14);
cout <<"\t\t Press ";
SetConsoleTextAttribute(h, 15);
cout <<"Backspace key to Go Back: ";

while (ch = getch())
{
    if (ch == '\b')
    {
        break;
    }
    else if (isspace(ch))
    {
        buyVehicleCustomerCount++;
        temp = stoi(vehicleStock[index]);
        temp--;
        vehicleStock[index] = to_string(temp);
        cout << endl << endl;
        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Vehicle has been purchased. Please pay the price
according to your payment"<< endl;
        cout <<"\t\t method and Get your Vehicle from the
Showroom..."<< endl << endl;
```

```
                    SetConsoleTextAttribute(h, 8);
                    cout <<"\t\t\t Press any key to Continue...";
                    getch();
                    return "Thanks for Purchasing!! Get your Vehicle and Enjoy";
                }
            }
        }
}


void _customerViewProductsMenu (int productCount, string productName[], string
productPrice[])
{
        if (productCount == 0)
        {
                SetConsoleTextAttribute(h, 12);
                cout << endl << endl;
                cout <<"\t\t No Vehicles Available..."<< endl;
        }
        else
        {
                SetConsoleTextAttribute(h, 13);
                cout <<"  "<< setw(25) << left <<"SrNo."<< setw(35) << left <<"Name"<<
setw(40) << left <<"Price"<< endl << endl;
                SetConsoleTextAttribute(h, 11);
                for (int x = 0; x < productCount; x++)
                {
                        cout <<"  "<< setw(25) << left << x + 1 << setw(35) << left <<
productName[x] << setw(40) << left << productPrice[x] << endl << endl;
```

```
            }
        }
        cout << endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Press any key to continue...";
        getch();
}


string chooseProductMenu (int customerCount, string customerUsername[], string
temporaryUsername, string customerName[], string customerPhoneNumber[], int productCount,
string productName[], string productPrice[], int buyProductCustomerCount, string
buyProductCustomerName[], string buyProductCustomerPhoneNumber[], string
buyProductName[], string buyProductPrice[])
{
        int index1, index2;
        bool checkProduct = false;
        bool checkCustomer = false;
        char ch;
        string name;

        if (productCount == 0)
        {
            return "No Product to buy...";
        }

        for (int i = 0; i < productCount; i++)
        {
            if (buyProductName[buyProductCustomerCount] == productName[i])
```

```
            {
                index2 = i;
                checkCustomer = true;
            }
        }

        if (checkCustomer == true)
        {
            SetConsoleTextAttribute(h, 14);
            cout <<"\t You have already Selected this Product: ";
            SetConsoleTextAttribute(h, 13);
            cout <<"  "<< setw(40) << left <<"Name"<< setw(40) << left <<"Price"<< endl
<< endl;

            SetConsoleTextAttribute(h, 11);
            cout <<"  "<< setw(40) << left << productName[index2] << setw(40) << left <<
productPrice[index2] << endl << endl<< endl;

            SetConsoleTextAttribute(h, 14);
            cout <<"\t\t If you are satisfied with this Product Press ";
            SetConsoleTextAttribute(h, 15);
            cout <<"Backspace to Return,"<< endl ;
            SetConsoleTextAttribute(h, 14);
            cout <<"\t\t If you want to change the Product Press ";
            SetConsoleTextAttribute(h, 15);
            cout <<"Enter or Space to Change: ";
            while(ch = getch())
            {
                if (ch == '\b')
```

```
                {
                        cout << endl;
                        return "Choose the Payment Method Now...";
                }
                else if (isspace(ch))
                {
                        cout << endl << endl;
                        goto loop;
                }
        }
}

loop:
for (int x = 0; x < customerCount; x++)
{
        if (temporaryUsername == customerUsername[x])
        {
                index1 = x;
        }
}

SetConsoleTextAttribute(h, 14);
cout <<"\t Enter the Name of the Product you want to Buy: ";
getline (cin, name);

for (int x = 0; x < productCount; x++)
{
        if (name == productName[x])
```

```
            {
                index2 = x;
                checkProduct = true;
            }
        }

        if (checkProduct == false)
        {
            return "No Product exists with this Name...";
        }
        else if (checkProduct == true)
        {
            cout << endl;
            SetConsoleTextAttribute(h, 14);
            cout <<"\t The Product available with this Name is: "<< endl << endl;
            SetConsoleTextAttribute(h, 13);
            cout <<"  "<< setw(40) << left <<"Name"<< setw(40) << left <<"Price"<< endl
<< endl;

            SetConsoleTextAttribute(h, 11);
            cout <<"  "<< setw(40) << left << productName[index2] << setw(40) << left <<
productPrice[index2] << endl << endl;

            SetConsoleTextAttribute(h, 14);
            cout <<"\t\t If you want to buy above Product Press ";
            SetConsoleTextAttribute(h, 15);
            cout <<"Enter or Space to Proceed,"<< endl ;
            SetConsoleTextAttribute(h, 14);
            cout <<"\t\t Press ";
```

```
            SetConsoleTextAttribute(h, 15);
            cout <<"Backspace key to Go Back: ";

            while (ch = getch())
            {
                if (ch == '\b')
                {
                    break;
                }
                else if (isspace(ch))
                {
                    buyProductCustomerName[buyProductCustomerCount] =
customerName[index1];
                    buyProductCustomerPhoneNumber[buyProductCustomerCount] =
customerPhoneNumber[index1];
                    buyProductName[buyProductCustomerCount] =
productName[index2];
                    buyProductPrice[buyProductCustomerCount] =
productPrice[index2];
                    cout << endl;
                    return "Product Selected Successfully. Please Select the payment
method now!!";
                }
            }
        }
}
```

string productBillMenu (int productCount, string productName[], string productStock[], int &buyProductCustomerCount, string buyProductCustomerName[], string

```
buyProductCustomerPhoneNumber[], string buyProductName[], string buyProductPrice[], string
buyProductPayment[])
{
        int index, temp;
        bool checkCustomer = false;
        bool checkPayment = false;
    char wall = 219, ch;
    int i = 33, j = 18;
        for (int i = 0; i < productCount; i++)
        {
                if (buyProductName[buyProductCustomerCount] == productName[i])
                {
                        index = i;
                        checkCustomer = true;
                }
        }

        if (buyProductPayment[buyProductCustomerCount] == "Immediate Cash Payment" ||
buyProductPayment[buyProductCustomerCount] == "Payment by Credit Card" ||
buyProductPayment[buyProductCustomerCount] == "Payment By Bank")
        {
                checkPayment = true;
        }
        if (checkCustomer == false)
        {
                return "Please Choose the Product First...";
        }
        if (checkPayment == false)
        {
```

```
                return "Please Choose the Payment Method First...";
        }
    char bill[26][100] =
    {

"wwww
```

```
    "ww ------------------------------------------------------------------------------------------ ww",
    "ww                                                                                          ww",

"wwww
```

```
    }
}
cout << endl;
j++;
}

SetConsoleTextAttribute(h, 10);
gotoxy(44, 24);
        cout << buyProductCustomerName[buyProductCustomerCount];
gotoxy(53, 26);
        cout << buyProductCustomerPhoneNumber[buyProductCustomerCount];
gotoxy(53, 31);
        cout << buyProductName[buyProductCustomerCount];
gotoxy(54, 33);
        cout << buyProductPayment[buyProductCustomerCount];
gotoxy(106, 38);
        cout << buyProductPrice[buyProductCustomerCount];

        cout << endl << endl << endl << endl << endl;
        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t If you want to buy above Product Press ";
        SetConsoleTextAttribute(h, 15);
        cout <<"Enter or Space to Proceed,"<< endl ;
        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Press ";
        SetConsoleTextAttribute(h, 15);
        cout <<"Backspace key to Go Back: ";
```

```
            while (ch = getch())
            {
                if (ch == '\b')
                {
                    break;
                }
                else if (isspace(ch))
                {
                    buyProductCustomerCount++;
                    temp = stoi(productStock[index]);
                    temp--;
                    productStock[index] = to_string(temp);
                    cout << endl;
                    SetConsoleTextAttribute(h, 14);
                    cout <<"\t\t Product has been purchased. Please pay the price
according to your payment"<< endl;
                    cout <<"\t\t  method and Get your Product from the
Showroom..."<< endl;


                    cout << endl << endl;
                    SetConsoleTextAttribute(h, 8);
                    cout <<"\t\t Press any key to Continue...";
                    getch();
                    return "Thanks for Purchasing!! Get your Product and Enjoy";
                }
            }
        }
}
```

```
string _addCommentsMenu (int customerCount, string customerUsername[], string
customerName[], string temporaryUsername, int &customerCommentsCount,  string
commentingCustomerName[], string commentingCustomerUsername[], string
customerComments[])
{
	int index;
	for (int x = 0; x < customerCount; x++)
	{
		if (temporaryUsername == customerUsername[x])
		{
			index = x;
		}
	}

	commentingCustomerName[customerCommentsCount] = customerName[index];
	commentingCustomerUsername[customerCommentsCount] = customerUsername[index];

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Add Feedbacks or comments... "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\tEnter: ";
	getline(cin, customerComments[customerCommentsCount]);
	customerCommentsCount++;
	return "Your message has been delivered, Successfully...";
}
```

```
void _serviceRegistrationMenu (int customerCount, string customerUsername[], string
customerName[], string temporaryUsername, int &customer4ServiceCount,  string
customer4ServiceName[], string customer4ServiceUsername[], string serviceVehicleMake[],
string serviceVehicleName[])
{
	int index;
	for (int x = 0; x < customerCount; x++)
	{
		if (temporaryUsername == customerUsername[x])
		{
			index = x;
		}
	}

	customer4ServiceName[customer4ServiceCount] = customerName[index];
	customer4ServiceUsername[customer4ServiceCount] = customerUsername[index];

	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t Add the following details for Registration... "<< endl << endl;
	SetConsoleTextAttribute(h, 11);
	cout <<"\t\t\t Enter your Vehicle Make: ";
	getline(cin>>ws, serviceVehicleMake[customer4ServiceCount]);
	cout <<"\t\t\t Enter your Vehicle Name: ";
	getline(cin>>ws, serviceVehicleName[customer4ServiceCount]);
	cout << endl << endl;

	customer4ServiceCount++;

	SetConsoleTextAttribute(h, 15);
```

```
	cout <<"\t Registration Completed Successfully!!"<< endl << endl;
	cout <<"\t Note: "<< endl;
	SetConsoleTextAttribute(h, 14);
	cout <<"\t\t The timings of Service Centre are from ";
	SetConsoleTextAttribute(h, 15);
	cout <<"8:00 AM to 4:00 PM.";
	SetConsoleTextAttribute(h, 14);
	cout <<" Please assure your presence"<< endl;
	cout <<"\t\t in the service centre between the mentioned time. Also, we provide various service "<< endl;
	cout <<"\t\t packages to our customers. For regular customers, discounts are also available. To "<< endl;
	cout <<"\t\t avail these packages, Contact us in the Service Centre.";
	cout << endl << endl;

	SetConsoleTextAttribute(h, 8);
	cout <<"\t\t Press any key to continue...";
	getch();
}


void _driveRegistrationMenu (int customerCount, string customerUsername[], string customerName[], string temporaryUsername, int &customer4DriveCount, string customer4DriveName[], string customer4DriveUsername[], string customer4DriveVehicle[], string vehicles4Drive[])
{
	int index1 = customerCount , temp;
	string index2;
	for (int x = 0; x < customerCount; x++)
	{
```

```
            if (temporaryUsername == customerUsername[x])
            {
                index1 = x;
            }
        }

        customer4DriveName[customer4DriveCount] = customerName[index1];
        customer4DriveUsername[customer4DriveCount] = customerUsername[index1];

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Select Vehicles from the following for Testing the Drive... "<< endl << endl;
        SetConsoleTextAttribute(h, 15);
        for (int x = 0; x < 7; x++)
        {
            cout << "\t"<< x+1 <<")   "<< vehicles4Drive[x] << endl;
        }
        cout << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t Enter your Choice: ";
        getline(cin>>ws, index2);

        cout << endl << endl;
        if (index2 == "1" || index2 == "2" || index2 == "3"|| index2 == "4"|| index2 == "5"|| index2
== "6"|| index2 == "7")
        {
            temp = stoi(index2);
            customer4DriveVehicle[customer4DriveCount] = vehicles4Drive[temp-1];
            customer4DriveCount++;
```

```
            SetConsoleTextAttribute(h, 15);
            cout <<"\t Registration Completed Successfully!!"<< endl << endl;
            cout <<"\t Note: "<< endl;
            SetConsoleTextAttribute(h, 14);
            cout <<"\t\t The timings For testing Drive are from ";
            SetConsoleTextAttribute(h, 15);
            cout <<"8:00 AM to 4:00 PM.";
            SetConsoleTextAttribute(h, 14);
            cout <<" Please assure your presence"<< endl;
            cout <<"\t\t between the mentioned time. Charges for testing the drive, you have
to deposit are "<< endl;
            cout <<"\t\t \"Rs 2000\"";

        }
        else
        {
            SetConsoleTextAttribute(h, 12);
            cout <<"\t\t Invalid input. Please try again!!";
        }
            cout << endl << endl;
            SetConsoleTextAttribute(h, 8);
            cout <<"\t\t Press any key to continue...";
            getch();
}
```

string customerAccountSettingMenu ()

```
{
   string option;

        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Choose you want to change.... "<< endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t 1) Name"<< endl << endl;
        cout <<"\t\t\t 2) Username"<< endl << endl;
        cout <<"\t\t\t 3) Password"<< endl << endl;
        cout <<"\t\t\t 4) Mobile Number"<< endl << endl;
        cout <<"\t\t\t 5) Go Back"<< endl << endl;
        SetConsoleTextAttribute(h, 8);
        cout <<"\t\t Enter Your Option (1-5): ";
        getline(cin, option);

        return option;
}


bool customerSignupMenu(int customerCount, string customerUsername[], string
customerPassword[], string customerName[], string customerPhoneNumber[])
{
        bool eligible;
        SetConsoleTextAttribute(h, 14);
        cout <<"\t\t Press Tab Key to Enter..."<< endl << endl << endl;
        SetConsoleTextAttribute(h, 11);
        cout <<"\t\t\t Enter your Full Name (3-10 characters): ";
        customerName[customerCount] = nameValidation(15,3);
```

```
	cout << endl;
	cout <<"\t\t\t Enter your User Name (4-10 characters): ";
	customerUsername[customerCount] = usernameValidation(10,4);
	cout << endl;
	cout <<"\t\t\t Enter your Password (4-10 characters): ";
	customerPassword[customerCount] = passwordValidation(10,4);
	cout << endl;
	cout <<"\t\t\t Enter your Mobile Number (11 digits): ";
	customerPhoneNumber[customerCount] = numberValidation(11,11);
	cout << endl;

	eligible = checkAccountEligibility (customerUsername, customerCount);
	return eligible;
}



bool checkUser (string username, string password, string usernameArr[], string passwordArr[],
int count)
{
	for (int x = 0; x <= count; x++)
	{
		if (usernameArr[x] == username && passwordArr[x] == password)
		{
			return true;
		}
	}
	return false;
```

```
}


bool checkAccountEligibility (string username[], int count)
{
	for (int x= 0; x < count; x++)
	{
		if (username[x] == username[count])
		{
			return false;
		}
	}
	return true;
}


bool checkEligibility (string array[], string value, int count)
{
	for (int x= 0; x <= count; x++)
	{
		if (array[x] == value)
		{
			return false;
		}
	}
	return true;
}
```

```
string numberValidation(int maximum, int minimum)
{
    string item = "";
    int count = 0;
    char ch;

    while (ch = getch())
    {
            if (isdigit(ch) && count <= maximum)
            {
        cout << ch;
                    item += ch;
        count++;
            }
      else if (GetAsyncKeyState(VK_TAB) && count >= minimum)
      {
        break;
      }
      else if (ch == '\b')
      {
        if (!item.empty())
        {
            cout << "\b \b";
            item.pop_back();
            count--;
        }
      }
```

```
    }
    item += '\0';
            return item;

}



string usernameValidation (int maximum, int minimum)
{
            string item = "";
    int count = 0;
    char ch;

    while (ch = getch())
    {
                if (IsCharAlphaNumeric(ch) && count <= maximum)
                {
            cout << ch;
                        item += ch;
            count++;
                }
        else if (GetAsyncKeyState(VK_TAB) && count >= minimum)
        {
            break;
        }
        else if (ch == '\b')
        {
            if (!item.empty())
```

```
            {
                cout << "\b \b";
                item.pop_back();
                count--;
            }
        }
    }
    item += '\0';
            return item;
}

string passwordValidation (int maximum, int minimum)
{
            string item = "";
    int count = 0;
    char ch;

    while (ch = getch())
    {
                if (IsCharAlphaNumeric(ch) && count <= maximum)
                {
            cout <<"*";
                        item += ch;
            count++;
                }
        else if (GetAsyncKeyState(VK_TAB) && count >= minimum)
        {
            break;
```

```
        }
        else if (ch == '\b')
        {
            if (!item.empty())
            {
                cout << "\b \b";
                item.pop_back();
                count--;
            }
        }
    }
    item += '\0';
            return item;
}
string nameValidation (int maximum, int minimum)
{
            string item = "";
    int count = 0;
    char ch;

    while (ch = getch())
    {
                if (isalpha(ch) && count <= maximum)
                {
            cout << ch;
                        item += ch;
            count++;
                }
```

```
        else if (GetAsyncKeyState(VK_TAB) && count >= minimum)
        {
            break;
        }
        else if (ch == '\b')
        {
            if (!item.empty())
            {
                cout << "\b \b";
                item.pop_back();
                count--;
            }
        }
    }
    item += '\0';
            return item;
}
void getUsersData (string filename, int &userCount, string name[], string username[], string
password[], string phoneNumber[], string CNIC[], string salary[])
{
    fstream file;
    file.open(filename, ios::in);
    string line;

    if (file.is_open())
    {
        while (!(file.eof()))
        {
```

```
            getline(file, line);
            if (line != "")
            {
                name[userCount] = getField (line, 1);
                username[userCount] = getField (line, 2);
                password[userCount] = getField (line, 3);
                phoneNumber[userCount] = getField (line, 4);
                CNIC[userCount] = getField (line, 5);
                salary[userCount] = getField (line, 6);
                userCount++;
            }
        }
    }
    file.close();
}
void getCustomerData (string filename, int &userCount, string name[], string username[], string
password[], string phoneNumber[])
{
    fstream file;
    file.open(filename, ios::in);
    string line;

    if (file.is_open())
    {
        while (!(file.eof()))
        {
            getline(file, line);
            if (line != "")
```

```
            {
                name[userCount] = getField (line, 1);
                username[userCount] = getField (line, 2);
                password[userCount] = getField (line, 3);
                phoneNumber[userCount] = getField (line, 4);
                userCount++;
            }
        }
    }
    file.close();
}


void getVehiclesData (string filename, int &vehicleCount, string make[], string name[], string
stock[], string price[])
{
    fstream file;
    file.open(filename, ios::in);
    string line;

    if (file.is_open())
    {
        while (!(file.eof()))
        {
            getline(file, line);
            if (line != "")
            {
                make[vehicleCount] = getField (line, 1);
```

```
            name[vehicleCount] = getField (line, 2);
            stock[vehicleCount] = getField (line, 3);
            price[vehicleCount] = getField (line, 4);
            vehicleCount++;
        }
    }
  }
  file.close();
}
void getProductsData (string filename, int &productCount, string name[], string stock[], string
price[])
{
  fstream file;
  file.open(filename, ios::in);
  string line;

  if (file.is_open())
  {
    while (!(file.eof()))
    {
      getline(file, line);
      if (line != "")
      {
        name[productCount] = getField (line, 1);
        stock[productCount] = getField (line, 2);
        price[productCount] = getField (line, 3);
        productCount++;
      }
```

```
        }
    }
    file.close();
}
void getVehicleServiceData (string filename, int &customer4ServiceCount, string
customerName[], string customerUsername[], string vehicleMake[], string vehicleName[])
{
    fstream file;
    file.open(filename, ios::in);
    string line;

    if (file.is_open())
    {
        while (!(file.eof()))
        {
            getline(file, line);
            if (line != "")
            {
                customerName[customer4ServiceCount] = getField (line, 1);
                customerUsername[customer4ServiceCount] = getField (line, 2);
                vehicleMake[customer4ServiceCount] = getField (line, 3);
                vehicleName[customer4ServiceCount] = getField (line, 4);
                customer4ServiceCount++;
            }
        }
    }
    file.close();
}
```

```
void getTestDerivesData (string filename, int &customer4TestDriveCount, string
customerName[], string customerUsername[], string vehicleName[])
{
    fstream file;
    file.open(filename, ios::in);
    string line;

    if (file.is_open())
    {
        while (!(file.eof()))
        {
            getline(file, line);
            if (line != "")
            {
                customerName[customer4TestDriveCount] = getField (line, 1);
                customerUsername[customer4TestDriveCount] = getField (line, 2);
                vehicleName[customer4TestDriveCount] = getField (line, 3);
                customer4TestDriveCount++;
            }
        }
    }
    file.close();
}
void getCommentsData (string filename, int &customerCommentsCount, string
customerName[], string customerUsername[], string comments[])
{
    fstream file;
    file.open(filename, ios::in);
    string line;
```

```
    if (file.is_open())
    {
        while (!(file.eof()))
        {
            getline(file, line);
            if (line != "")
            {
                customerName[customerCommentsCount] = getField (line, 1);
                customerUsername[customerCommentsCount] = getField (line, 2);
                comments[customerCommentsCount] = getField (line, 3);
                customerCommentsCount++;
            }
        }
    }
    file.close();
}
string getField (string data, int field)
{
    int commaCount = 1;
    string word;
    for (int x = 0; x < data.length(); x++)
    {
        if (data[x] == ',')
        {
            commaCount++;
        }
        else if (commaCount == field)
```

```
        {
            word = word + data[x];
        }
    }
    return word;
}
void storeUsersData (string filename, int userCount, string name[], string username[], string
password[], string phoneNumber[], string CNIC[], string salary[])
{
    fstream file;
            file.open(filename, ios::out);
            for (int x = 0; x < userCount; x++)
            {
                    file << name[x] << "," << username[x] << "," << password[x] << "," <<
phoneNumber[x] << "," << CNIC[x] << "," << salary[x] << endl;
            }
            file.close();
}
void storeCustomerData (string filename, int userCount, string name[], string username[], string
password[], string phoneNumber[])
{
    fstream file;
            file.open(filename, ios::out);
            for (int x = 0; x < userCount; x++)
            {
                    file << name[x] << "," << username[x] << "," << password[x] << "," <<
phoneNumber[x] << endl;
            }
            file.close();
```

```
}
void storeVehiclesData (string filename, int vehicleCount, string make[], string name[], string stock[], string price[])
{
    fstream file;
        file.open(filename, ios::out);
        for (int x = 0; x < vehicleCount; x++)
        {
            file << make[x] << "," << name[x] << "," << stock[x] << "," << price[x] << endl;
        }
        file.close();
}

void storeProductsData (string filename, int productCount, string name[], string stock[], string price[])
{
    fstream file;
        file.open(filename, ios::out);
        for (int x = 0; x < productCount; x++)
        {
            file << name[x] << "," << stock[x] << "," << price[x] << endl;
        }
        file.close();
}

void storeVehicleServiceData (string filename, int customer4ServiceCount, string customerName[], string customerUsername[], string vehicleMake[], string vehicleName[])
{
    fstream file;
```

```
        file.open(filename, ios::out);
        for (int x = 0; x < customer4ServiceCount; x++)
        {
                file << customerName[x] << "," << customerUsername[x] << "," <<
vehicleMake[x] << "," << vehicleName[x] << endl;
        }
        file.close();
}

void storeTestDerivesData (string filename, int customer4TestDriveCount, string
customerName[], string customerUsername[], string vehicleName[])
{
   fstream file;
        file.open(filename, ios::out);
        for (int x = 0; x < customer4TestDriveCount; x++)
        {
                file << customerName[x] << "," << customerUsername[x] << "," <<
vehicleName[x] << endl;
        }
        file.close();
}

void storeCommentsData (string filename, int customerCommentsCount, string
customerName[], string customerUsername[], string comments[])
{
   fstream file;
        file.open(filename, ios::out);
        for (int x = 0; x < customerCommentsCount; x++)
        {
```

```
                file << customerName[x] << "," << customerUsername[x] << "," << comments[x]
<< endl;
        }
        file.close();
}
void resizeConsole()
{
        COORD coord;
        SMALL_RECT rect;
        HWND console = GetConsoleWindow();
        coord.X = 168;
        coord.Y = 1080;
        rect.Left = 0;
        rect.Top = 0;
        rect.Right = 167;
        rect.Bottom = 1079;
        SetConsoleScreenBufferSize(GetStdHandle(STD_OUTPUT_HANDLE), coord);
        SetConsoleWindowInfo(GetStdHandle(STD_OUTPUT_HANDLE), TRUE, &rect);
        ShowWindow(console, SW_MAXIMIZE);
}
void gotoxy (int x, int y)
{
        COORD coordinates;
        coordinates.X = x;
        coordinates.Y = y;
        SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coordinates);
}
```

9. Weakness in the Application

- This application can not display the Details of the vehicle such as models, colors, etc.
- If any user has forgotten his username or password, he cannot recover it.
- Customer cannot buy the vehicle on installments

10. Future Directions

- Add the details of vehicles and products.
- Allow option for employee to attend the customer registered for vehicle.
- Allow employee to mark attendance through application.
- Allow managers to view the attendance of employees.
- Allow manager to add schedules of employees working times.
- Add buy vehicles on installments option.